VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines

AALHAD SAWANE, [TODO: institution], [TODO: country]

Short serverless functions spend most of their time waiting for an isolated environment to start or for packets to cross a long distance, while computers close to users sit idle. We ask whether idle consumer computers can answer requests that must finish within seconds. Two things make this hard. Owners switch machines off without warning, which destroys every prepared environment on them. And owners cannot be trusted with the code and data they run. Remote attestation, in which a machine proves to a verifier which software it booted, changes the second problem: an owner who controls the machine's software, power and network can still stop or delay it, but can no longer read or forge anything. What is left is a scheduling problem. Proving trust, handing a machine a customer's decryption key, and starting environments are slow; all of it is lost when the owner switches off; and every extra machine that holds a key is one more place it could leak from. We present Voss, a scheduler built for this problem. A planner, run every 30 seconds, decides which machines hold which keys and ready environments, with spare capacity for machines that disappear. A per-request step then picks a machine for each call and duplicates calls at risk. A machine is judged failed when work it accepted does not come back, not when it stops sending heartbeats, and a failed scheduler is replaced without any agreement protocol among volunteers. We describe two architectures, one global scheduler on provider hardware and geographic clusters with volunteer-hosted schedulers reached through a publish/subscribe broker. In a deterministic simulator calibrated on a TPM-equipped laptop, Voss keeps 99th-percentile latency under 3 s in 9 of 9 churn-and-adversary settings and 13 of 16 settings that vary the trust technology and load, where the strongest simple baseline does so in none. We also report the parts that did not pay off.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Security and privacy** → *Trusted computing*; • **Networks** → Network protocols.

Additional Key Words and Phrases: serverless computing, volunteer computing, remote attestation, scheduling, publish/subscribe, tail latency

ACM Reference Format:
Aalhad Sawane. 2026. VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines. *ACM Trans. Internet Technol.* 1, 1 (November 2026), 34 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Serverless platforms run small pieces of customer code, called *functions*, whenever a request arrives. Most of these functions finish in well under a second [48], so the useful work is small compared with what surrounds it. Before a function can run, the platform must create an isolated environment for it and load its language runtime. This *cold start* can take longer than the function itself. The request must also travel to wherever the function runs. Providers hide cold starts by keeping environments alive in their datacenters, which holds memory around the clock [27, 48]. They can do little about distance.

Meanwhile, millions of home and office computers sit close to users and idle for most of the day. Volunteer computing has used such machines for two decades [6, 7], but only for batch jobs with deadlines of hours. We ask whether volunteer machines can answer requests that must finish in a few seconds.

Author's address: Aalhad Sawane, [TODO: institution], [TODO: country], aalhadsawane@gmail.com.

2026. ACM 1533-5399/2026/11-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Two problems stand in the way. The first is **churn**: an owner can switch off, reboot or reclaim the machine at any moment, and everything the platform prepared on it disappears, including started environments, cached code and the secret keys it was given. For a function that runs for tens of milliseconds, preparing this state again costs far more than the function. The second is **distrust**: the owner controls the hardware and the operating system, so the owner can read the customer's code and data, return forged results, or walk away at the worst moment. The two problems feed on each other. The usual answer to distrust is to check a machine before using it, and that check takes time; churn throws the result away whenever a session ends.

Earlier work solves parts of this. Volunteer-computing systems run every task on several machines and compare the answers [6]. That multiplies cost, works only for tasks with one right answer, and still shows the input to every machine that runs it. Serverless schedulers avoid cold starts well [27, 48] but assume trusted machines that stay up. Confidential-computing designs protect customers from the platform operator [1, 9], but they run on provider servers and treat the trust check as a one-time setup step. And the usual way to notice a failed machine, waiting for its periodic "I am alive" message (a heartbeat), does not work against an owner who keeps sending heartbeats while discarding the work. Huang et al. call such a failure, where a component looks healthy to simple checks but fails the requests it serves, a *gray failure* [31].

Our starting point is that proving which software a machine runs changes what a hostile owner can do. Suppose the customer's code, and the scheduler's own code, run only inside a locked operating-system image, and a verifier has checked hardware-signed evidence that the machine booted exactly that image. This check is *remote attestation*. The owner can then no longer read, forge or change anything inside the image. What the owner keeps is the power switch and the network cable, so a hostile owner can stop the machine or delay its answers, which is what churn does anyway. The literature calls these *omission* and *timing* failures [10, 20]. A volunteer platform must tolerate them regardless, so the same mechanisms cover crashes, churn and malicious owners.

This leaves a scheduling problem with a specific shape. During one session a machine goes through three steps for a customer: it is attested; it receives the customer's decryption key from a key service the customer controls; and it starts environments for the customer's functions. All three are expensive, since a powered-off laptop needs 65 s to become an attested worker, thousands of times longer than reusing an environment that is already running. All three are lost when the session ends. And every machine that holds a customer's key is one more place the key could leak from if the trust assumptions fail. Choosing which machines go through these steps, and when, is a decision with a cost, and it cannot be made one request at a time.

We present Voss, a scheduler built around this problem (Figure 2). A *planner* runs every 30 seconds and decides which machines hold which customer's key and which functions they keep ready, keeping enough spare capacity that losing a few machines does not hurt. A per-request step then chooses a machine for each call and sends a duplicate when the chosen machine is likely to fail. A machine is judged failed when work it accepted does not come back, and a failed scheduler is replaced by a provider-run coordinator without asking volunteers to agree on anything. Signed receipts record who did which work. We describe two architectures: one global scheduler on provider hardware, and geographic clusters whose schedulers run on volunteer machines and are reached through a publish/subscribe broker.

Our contributions are: (1) an argument that attestation reduces hostile owners to omission and timing failures, and a statement of the scheduling problem that remains; (2) a two-part scheduler whose planner keeps capacity even if any f machines fail together; (3) an architecture for reaching schedulers on volunteer machines whose addresses change and cannot accept incoming connections, compared with a provider-hosted global scheduler; (4) failure detection and scheduler replacement that need no agreement among volunteers; (5) a deterministic, calibrated evaluation

whose hypotheses and decision rules were fixed before the runs, reported with the results that went against us.

Planning has the largest effect: it lowers 99th-percentile latency by 54–59% against provisioning machines only when requests arrive, and judging machines by returned work keeps 99.95% of requests successful when 20% of owners drop work, against 96.8% with heartbeats. Some components did not pay off, including geographic clusters and an exact solver, and a commercial-style datacenter reference is still faster. All results come from simulation and one calibration machine (Section 10.1).

2 BACKGROUND AND PROBLEM

Terms. A *tenant* is a customer of the platform: a developer or company that deploys functions. An *invocation* is one call of a function. The *provider* runs the platform. A *volunteer*, or *owner*, owns a machine that donates capacity, and a *worker* is such a machine while it runs our software. A *sandbox* is a small virtual machine (a microVM) that runs one function in isolation; it is *warm* when it has already started and sits idle, ready for a call. A *snapshot* is a saved copy of a started sandbox's memory, which can be restored faster than a sandbox can boot. A tenant's *key* is a secret decryption key that only the tenant controls.

2.1 Where the time goes

An invocation waits for several things in turn: choosing a region and a machine, waiting for a free slot on that machine, making the machine able to run this tenant's code, starting a sandbox, moving the input and output, and running. For functions that run for tens to hundreds of milliseconds, starting the sandbox dominates unless one is already warm. This is why platforms keep sandboxes alive after use or restore them from snapshots [8, 15, 27, 48]. A function can be in one of four states on a machine, from slowest to fastest start: **L0**, nothing on the machine, so the code and file system must be fetched and checked first; **L1**, code and file system on local disk, so a sandbox must boot and initialise; **L2**, a snapshot on local disk, so it is restored; **L3**, a warm sandbox in memory, so the request is simply handed to it. Each step up costs memory or disk, so choosing which functions to keep at which level on which machines is already a scheduling problem.

Volunteer machines add a cost that datacenters do not have. Before a machine may run tenant code it must prove which software it runs, and before it can decrypt a tenant's code it must receive that tenant's key. Figure 1 puts these costs on one scale, measured on a consumer laptop with a TPM, the security chip found on most modern PCs (Section 8.2). Handing a request to a warm sandbox takes milliseconds. Booting a new sandbox takes about 154 ms if its files are already in memory and 1.45 s if they must be read from the hard disk. Getting from power-off to an attested worker takes 65 s (median of 9 boots): 21 s for firmware, boot loader and kernel start, then 44 s until the verifier accepts the machine's proof. These numbers are upper bounds, because the laptop had a hard disk and we ran the image in a simplified mode. Even so, establishing trust costs thousands of times more than reusing a sandbox, so it has to be done before requests arrive, not while they wait.

The network is the second difference. A volunteer machine sits behind a home connection, often on Wi-Fi, with less upload than download capacity. In our model the download speed has a median of 100 Mbit/s and the upload speed a median of about 30 Mbit/s, with a wide spread across machines (Section 8.1). Code and file systems are 90 to 450 MB per function and snapshots 48 to 600 MB. A 450 MB file system takes about 36 s at 100 Mbit/s, and a 600 MB snapshot copied from a neighbour's 30 Mbit/s upload takes almost three minutes. Voss handles this in three ways. The planner prepares machines before demand arrives, so most transfers happen while no request waits. The per-request step charges each candidate the time to fetch what it is missing over that machine's own link, so a machine on a slow link loses to a machine that is ready. And workers fetch from a nearby worker

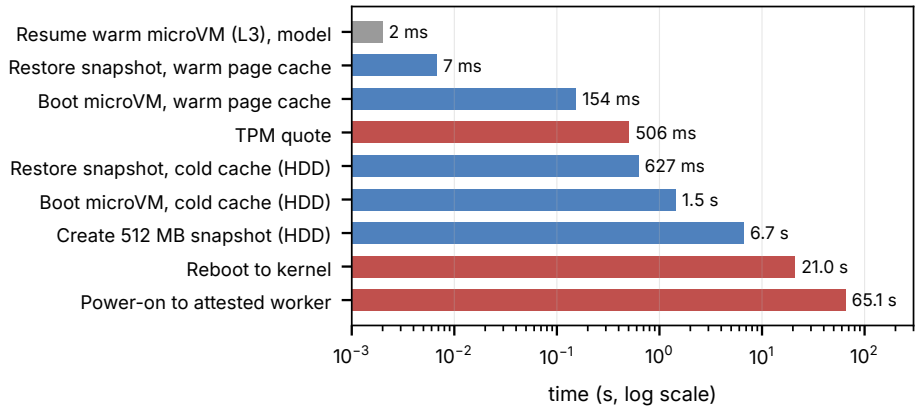


Fig. 1. Time to reach each state on the calibration laptop, on a log scale. Blue: sandbox start-up. Red: becoming an attested worker. Grey: model value, not measured. *Warm cache*: the file is already in memory; *cold*: it is read from the hard disk. *Reboot to kernel* is the physical machine's firmware, loader and kernel start.

before the provider's registry, while snapshots are never downloaded from the registry at all: a worker creates one locally after the first boot, or copies one from a nearby worker.

2.2 Proving which software a machine runs

We use the vocabulary of the IETF attestation architecture [12]. While a machine boots, each stage (firmware, boot loader, kernel) is hashed into registers of its TPM, so the final register values are a fingerprint of everything that ran [50]. When asked, the TPM signs these values together with a fresh random number chosen by the checker. The signed object is called a *quote*. The *verifier* checks the signature, follows the certificate chain back to the chip's manufacturer, and compares the fingerprint with the values expected for our image. A *key service*, run for the tenant, then releases a key only to machines the verifier accepted [45]. Two facts matter for scheduling. A quote shows what ran at one moment, so the evidence goes stale and must be refreshed. And attestation says nothing about availability: an attested machine can still stop working.

2.3 How volunteer machines fail

A volunteer machine alternates between being off and a *session*, an interval during which it runs our image. Session lengths are heavy-tailed and follow the time of day [34]. A session ends either with notice from the owner or abruptly, by reboot or power loss, and sometimes for a whole city at once. We use the standard classes of failure [10, 20]: a *crash* stops the machine; an *omission* is a result that never arrives; a *timing* failure is a result that arrives too late. Churn produces crashes. An owner who cannot forge anything can still choose when to drop or delay work, and can do so selectively while the machine keeps sending heartbeats. Heartbeat-based detectors, including adaptive ones such as the φ-accrual detector [29], cannot see this.

2.4 The problem

Requests arrive from clients in many places. Each names a tenant and a function, belongs to a service class (synchronous, where the client waits; asynchronous; or low-priority background work), and carries a deadline. Each function is marked *idempotent* or not; a function is idempotent

if calling it twice has the same effect as calling it once. The platform retries and duplicates only idempotent functions, and for the others a lost result is reported as a failure.

Asking for idempotence is not new. Every major platform already asks it of developers, although the guarantee differs by how a function is called. AWS Lambda retries a failed asynchronous invocation twice by default and warns that a function may receive the same event more than once [3]; for a direct synchronous call it does not retry, and leaves the retry, with the same warning, to the caller [4]. Google's event-driven functions run at least once per event, functions that are retried must be idempotent, and plain HTTP functions are not retried by the platform [28]. Azure documents at-least-once delivery for Event Hubs triggers and asks for consumers that tolerate duplicates [38], and Cloudflare Queues deliver at least once [16]. So delivery is at least once on event-driven and asynchronous paths, and the client retries on request–response paths. On both, a duplicate eventually arrives, so developers are told to write idempotent code. Voss relies on the same contract, and goes further than these platforms by also retrying and duplicating synchronous calls, because failures on volunteer machines are common.

The machines form a changing population, each with a number of slots for concurrent invocations, memory, disk, a location and a session that can end at any time. The scheduler decides which machines to attest and give which tenant's key, which functions to keep at which level where, and which machine runs each invocation. We pursue four goals. **G1:** an owner who controls the machine's software, power and network learns no tenant secret and forges no result. **G2:** 99th-percentile latency stays within the target although sessions end at any time. **G3:** a machine that accepts work and drops it, or a scheduler that stalls, is bypassed within seconds, without relying on the machine's own reports. **G4:** a machine cannot claim credit for work it did not do. We do not defend against physical attacks on the TPM, flaws in our image, side channels, or failure of the provider's own services (Section 4.1).

3 SYSTEM DESIGN

This section describes who takes part, what each party is trusted with, and what happens to one request. Three choices shape the design. Everything that touches tenant keys, or makes scheduling decisions, runs inside the attested image, so the rest of the system only has to cope with machines that stop or answer late (Section 4). Trust, keys and warm sandboxes are prepared ahead of demand, because they take seconds to minutes to establish (Section 5). And a machine is judged by whether the work it accepted comes back, as seen by the router or the scheduler, not by what the machine says about itself (Section 7).

3.1 What the provider stores, and why a compromised provider learns little

A tenant encrypts its code bundle with its own key before uploading it. The provider's registry, the disks of workers that cache the bundle, and any snapshot of a running sandbox hold only this ciphertext; snapshots are encrypted with the same key when they are written. The key itself never passes through the provider: it travels from the tenant's key service directly to an attested worker, over a channel that ends inside the attested image, and stays in that worker's memory until the session ends. As a result, an attacker who takes over the provider's registry, database, router or broker obtains encrypted bundles and request metadata, but no tenant code and no tenant data.

3.2 The parties

Figure 2 shows the components for the clustered architecture of Section 3.3.

Provider services are assumed reliable and are trusted to keep running and to keep correct records, but never with tenant keys. The *router* is the public entry point. A client sends its request to the nearest router. The router chooses a cluster, sends the request's metadata (not its input) to

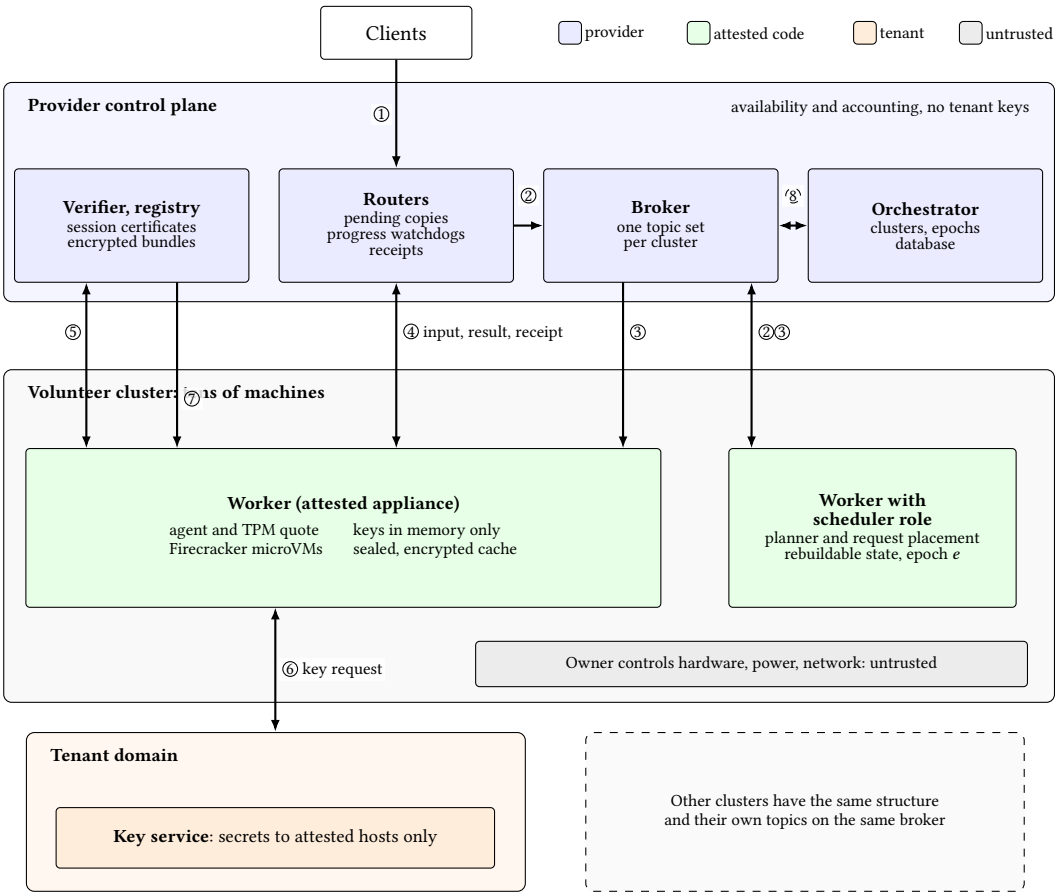


Fig. 2. The Voss architecture with geographic clusters. Numbers follow one invocation: (1) a client calls a router; (2) the router sends the request's metadata to the cluster's scheduler through the broker; (3) the scheduler sends the assignment to a worker; (4) the router sends the input and the worker returns the result and a signed receipt; (5) the worker proves itself to the verifier; (6) it asks the tenant's key service for the key; (7) it fetches the encrypted code from the registry. (8) The orchestrator appoints schedulers, off the request path.

that cluster's scheduler, and keeps its own copy of the request until a worker confirms it has started. If the scheduler does not answer within 1 s, the router resends the request or sends it to another cluster. When the worker is chosen, the router streams the input to it, receives the result and returns it to the client, and checks and stores the receipt the worker signs for that work. The *orchestrator* decides which workers form each cluster and appoints each cluster's scheduler. The *verifier* checks attestation evidence and acts as the certificate authority: it issues each attested worker a session certificate and revokes it when the session ends (Section 4). The *registry* stores encrypted code bundles and file systems, and the *broker* carries messages between a cluster's machines (Section 6).

Tenants run, or entrust to a service they choose, a *key service* that holds their keys and releases one only to a worker the verifier has accepted. The provider does not run it.

Volunteer machines reboot into the *attested image*: a locked, measured operating system containing the worker software, the Firecracker microVM monitor [2] and the scheduler code. Each worker runs every invocation in its own microVM and keeps an encrypted cache of code and snapshots. While the image runs the owner cannot use the machine, so donation means a scheduled or dedicated session rather than background use of a desktop. The owner still controls power, network and firmware settings, which is why a worker can stop or answer late even though its code cannot misbehave.

3.3 Two architectures

We designed and compared two ways to organise scheduling.

One global scheduler. A single scheduler, run on provider hardware, plans and places for every worker. Workers open connections to it at a stable address, and its failures are handled by ordinary replication inside the provider. This is simple, and it lets the planner pool all machines when deciding where keys and warm sandboxes go.

Geographic clusters. The orchestrator groups workers into clusters of tens of machines by network distance, and each cluster has its own scheduler. Each planning problem stays small and the scheduler is close to its workers. The scheduler runs on one of the cluster's own volunteer machines: the scheduler code is part of the image, so any worker can take the role without booting anything, and the provider runs no per-cluster server. The price is that a scheduler can now fail like any volunteer machine, which Section 7 handles, and that it is hard to reach, which Section 6 handles with a publish/subscribe broker.

Both use the same planner, per-request step and workers. Section 9.6 compares them: at the scale we can simulate, the global scheduler was faster.

3.4 Life of an invocation

The router sends the request's metadata to the chosen cluster's scheduler, which picks a worker and sometimes a second one for a duplicate copy (a *hedge*). The router then sends the input to the worker over a connection that ends inside the attested image, so the scheduler never sees inputs, results or keys. A worker without the tenant's key first requests it from the key service, then starts the function at whatever level it has reached and runs it. The result returns to the router with a signed receipt (Section 7.3). Each part of the latency is logged separately, which lets Section 9 attribute delays to their causes.

3.5 Sessions

A worker becomes usable only after booting and passing attestation, which takes 65 s from power-on on our laptop. When the session ends, with or without notice, every key and warm sandbox on the worker is gone. Cached bundles and snapshots stay on disk, still encrypted, and are usable again after the next attestation and key release. So within a session each function on a worker is at one of the levels L0 to L3 of Section 2, and separately the worker either holds or does not hold the tenant's key. The simulator reads these two facts, and their costs, from a data file for each trust technology (Section 4.5).

3.6 Rules for messages and state

Machines interact only by typed messages, and every connection is mutual TLS: both ends present certificates, and a worker or scheduler presents the session certificate the verifier issued after attestation. The provider's database is private to the provider services and is never on the request path, because a database round trip would add to every request's latency. State lives where it is created: queues, caches and keys on workers, unassigned requests at routers, estimates in the

Table 1. Adversaries, what stops them, and what they can still do.

Adversary	Can	Stopped by	Can still
Volunteer owner	control hardware, power and network; boot modified software; fake a TPM; replay, delay or drop messages	measured boot and attestation; manufacturer-certified TPM keys; a locked image with no login; a disk key sealed to the measurement	stop or delay
Network attacker	read and change traffic	mutual TLS ending inside attested parties; signed messages	see traffic volume
Other tenants	run malicious function code	microVM isolation	side channels

scheduler. After a scheduler is replaced, its successor rebuilds its view by asking the workers (Section 7.2).

4 TRUST AND ITS COST

The design rests on one claim: attestation turns a hostile owner into an unreliable one. This section states the claim and its limits, describes how a machine earns a certificate and a tenant's key, and explains why we still limit how many machines hold each key.

4.1 Threat model

We trust the CPU, its TPM and its memory protection; our measured image; each tenant's own key service; and the provider's services, to keep running and keep correct records (Table 1). We do not defend against laboratory-grade physical attacks such as voltage glitching of firmware TPMs [33], against flaws in our image, against side channels and traffic analysis, or against failure of the provider's services. An owner who can read the memory of a running machine, by a cold-boot attack or by probing the memory bus, is also outside the claim below unless the machine encrypts its memory.

Property 1. Under these assumptions, everything a worker or scheduler machine does that others can observe, and that its owner can cause through software, is a crash, a dropped message or a delayed message. The argument is short. The owner cannot run other software, because it would fail attestation and receive neither a certificate nor a key. The owner cannot read or change the image's memory or its encrypted disk. Results and receipts are signed with a key that never leaves the image. What remains is the power switch, the network path, and the choice of when to stop answering. So the mechanisms that churn forces on us anyway also cover malicious owners, and running every task on several machines is no longer needed to check results.

Other mechanisms fall short: memory encryption alone attests nothing, voting over several copies triples the cost, and proofs that let a client verify a computation are far too slow for millisecond functions. Confidential virtual machines exist only on server processors, so we treat them as one trust technology among several (Section 4.5). A blockchain would protect against no adversary here, because a receipt signed by an attested key can already be checked by anyone (Section 7.3).

4.2 How a machine becomes an attested worker

The image is a signed kernel with a read-only, integrity-checked root file system, the worker software and the scheduler code. It has no owner login, and the disk partition that holds the cache is encrypted with a key that the TPM releases only if the machine booted the expected image. A

session starts when the owner reboots into the image and ends on reboot, power loss, a request to leave, or failure.

Figure 3 shows the exchange, which follows Keylime [45]. The worker creates a fresh *session key pair* inside the image and includes a hash of the public key in the quote. The verifier checks the quote, the certificate chain and the boot record. It also acts as the certificate authority: if everything matches, it issues a *session certificate* that names the machine, carries the session public key and a trust score, and expires when the session expires. Periodic re-quotes keep it valid during the session. When the session ends, or a re-quote fails, the verifier revokes the certificate and tells the routers, the scheduler and the key services. Every connection in the system is mutual TLS with this certificate, so both ends of every connection are either a provider service or an attested image. Two consequences matter for scheduling. Trust is *perishable*: it lasts one session at most. And identity costs hardware: a worker is identified by a hash of its TPM's endorsement key, so creating many identities requires many machines [23].

4.3 How a worker gets a tenant's key

Attestation shows that a machine runs our image, not that it may see a particular tenant's data. Each tenant's key service applies the tenant's policy, for example "accept image versions at or above this one". Giving worker j tenant t's key takes one round trip: the worker presents its certificate over mutual TLS, the key service checks the policy and returns the key encrypted to the worker's session public key, and the worker keeps it in memory only. We say the worker is then *provisioned* for the tenant. Because bundles and snapshots are encrypted at rest (Section 3.1), any worker may store and forward them, but only provisioned workers can use them. Snapshots contain the decrypted memory of a running sandbox, so only provisioned workers create them, and they are encrypted with the tenant's key before being written.

Which party the tenant must trust for confidentiality depends on its policy. If the key service accepts any certificate the verifier issued for an approved image, the tenant trusts the verifier as well. If the policy requires a fresh quote, the key service checks the quote itself and does not depend on the verifier, at the cost of one more quote per release. The simulator charges a fixed, measured cost for the policy check.

4.4 Key exposure, and why we limit it

Every machine that holds a tenant's key is one more place the key could leak from if our assumptions fail. We measure this as the tenant's *key exposure*,

$$E_t = \sum_{j \text{ holds } t\text{'s key}} (1 - a_j), \tag{1}$$

where $a_j \in [0, 1]$ is machine j's *trust score*: a value set by policy for its trust technology (0.8 for the TPM image on consumer hardware, 0.95 for a confidential virtual machine), reduced if its firmware is not on a known-good list. Exposure is a count of key holders, each discounted by how well protected it is. It is not a probability of compromise, and we report it next to latency rather than folding it into one number. Tenants choose how much exposure matters to them, and can set a minimum trust score below which a machine never gets their key.

If attested machines are honest, why limit exposure at all? Three reasons. First, honesty holds only under the assumptions above, and on consumer hardware the assumptions we exclude are the realistic ones: reading the memory of a running machine that does not encrypt memory, flaws in TPM firmware [33], and bugs in our image. Limiting exposure is the defence for the case where attestation is not enough. Second, the two possible failures are not alike. A forged result harms one invocation and can be caught by re-running it. A leaked key is silent and permanent: it decrypts

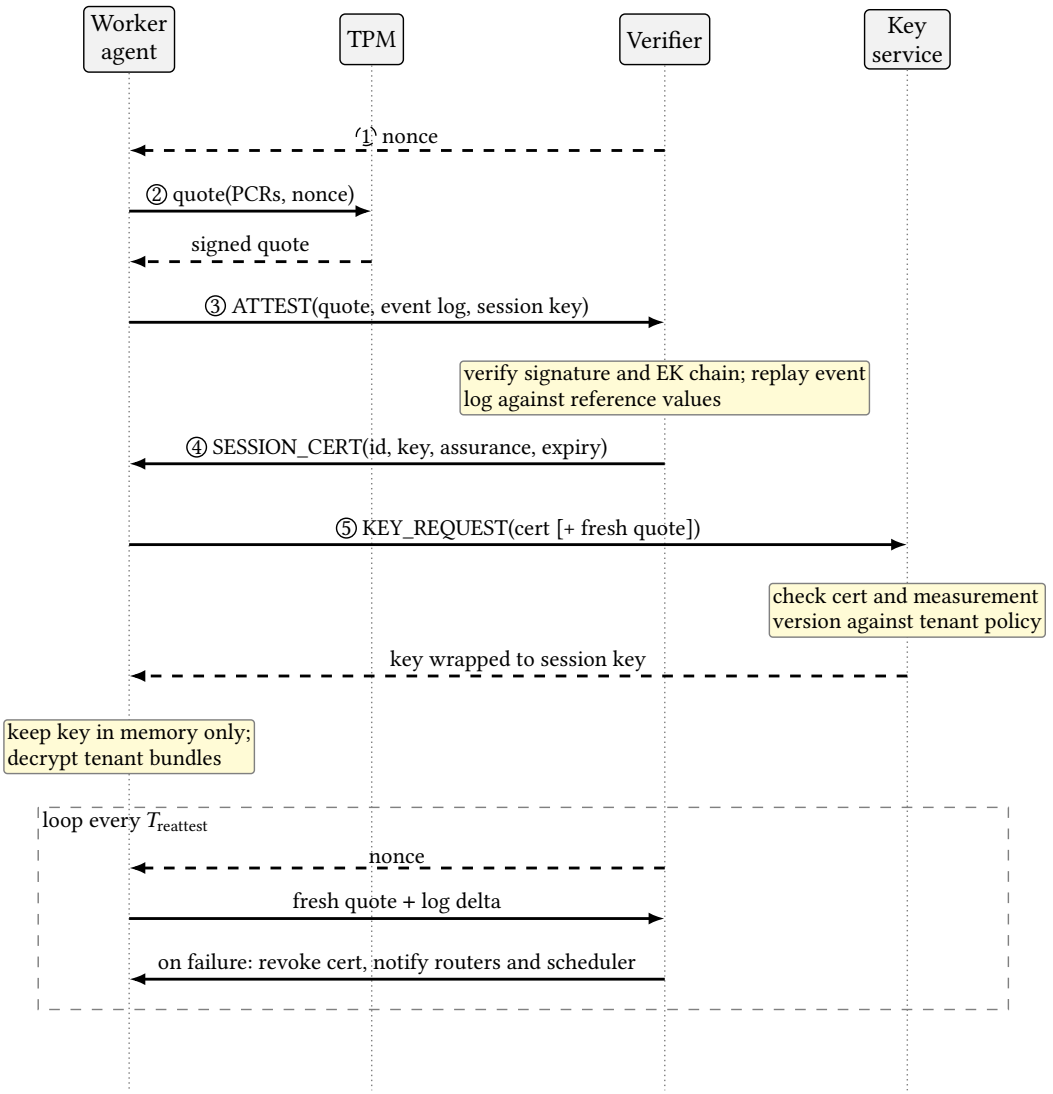


Fig. 3. Attestation and key release for one worker. The session key is bound into the quote, so the certificate belongs to this boot of the image only.

every bundle and snapshot of that tenant, past and future, wherever the ciphertext is stored, and reading memory passively is the easiest physical attack. Third, it costs almost nothing. Removing the exposure term from Voss changes 99th-percentile latency by 0.02 s (95% interval up to 0.06 s, including zero) while raising exposure by 2.6 (Section 9.3). Dropping it would weaken the defence without making anything faster.

4.5 Trust technologies

The scheduler does not depend on how a machine establishes trust. For each *trust technology* a data file tells the simulator how long attestation takes, how long it lasts, what each start-up level costs

Table 2. How often the scheduler's inputs change, and how long its decisions last.

Input or decision	Changes or lasts	Learned from
Busy slots, what is cached on each worker	milliseconds; reported every 2 s	worker reports
Request rate per tenant and function	meaningful over about a minute	request counts
Session length, chance a worker drops work	learned over hours	past sessions and outcomes
Assign one request	one request (milliseconds to seconds)	current state
Keep a sandbox warm	minutes	forecast of the request rate
Release a key to a worker	the rest of the session	forecasts of rate and session survival

and what trust score the machine carries. We model five: the TPM-attested image on consumer hardware, with costs we measured (Section 8.2); two kinds of confidential virtual machine on server processors, with costs from the literature [5, 30, 37, 39]; a measured hypervisor started under the owner's operating system; and no attestation, which the baselines use.

5 SCHEDULING

Section 4 made trust a resource with a price. This section decides who should hold it and when, and then which machine runs each request. We first explain why these are two separate decisions.

5.1 Why two loops

The scheduler's inputs change at very different rates, and its decisions last for very different times (Table 2). How many requests are running on a worker changes every few milliseconds and reaches the scheduler every 2 s. A tenant's request rate is only meaningful averaged over a minute or more. How long sessions last can only be learned from many sessions, over hours. On the decision side, assigning a request to a worker lasts as long as that request. Keeping a sandbox warm lasts minutes. Releasing a key to a worker lasts the rest of its session, minutes to hours, and cannot be undone.

A decision whose effect outlasts the refresh period of its inputs has to be made from a forecast, ahead of time. A decision whose effect is short can react to the latest state. Assigning a request is of the second kind; releasing keys and warming sandboxes are of the first. Two further facts separate them. Whether demand is still covered if f machines disappear is a property of the whole set of machines holding a key, and so is the key exposure of Section 4.4: neither can be judged one request at a time. And preparing a machine takes seconds to minutes (Figure 1), far more than a request can wait.

So the scheduler has two parts that share one view of the cluster (Figure 4). The *planner* runs every 30 s, and earlier if demand or membership changes sharply. It decides which workers hold which tenant's key, which functions each keeps warm and how many, which snapshots stay on disk, and how many slots on each worker are reserved for each tenant. *Request placement* runs for every request and chooses a worker, mostly from those the planner prepared. The 30 s period is a design choice, not a measured optimum: plan actions take from a few hundred milliseconds to several seconds, so the period must be longer, and sessions last minutes to hours, so it must be much shorter. We did not sweep it. Section 9.3 tests whether the planner is needed by comparing it with provisioning on demand and with a keep-alive timer.

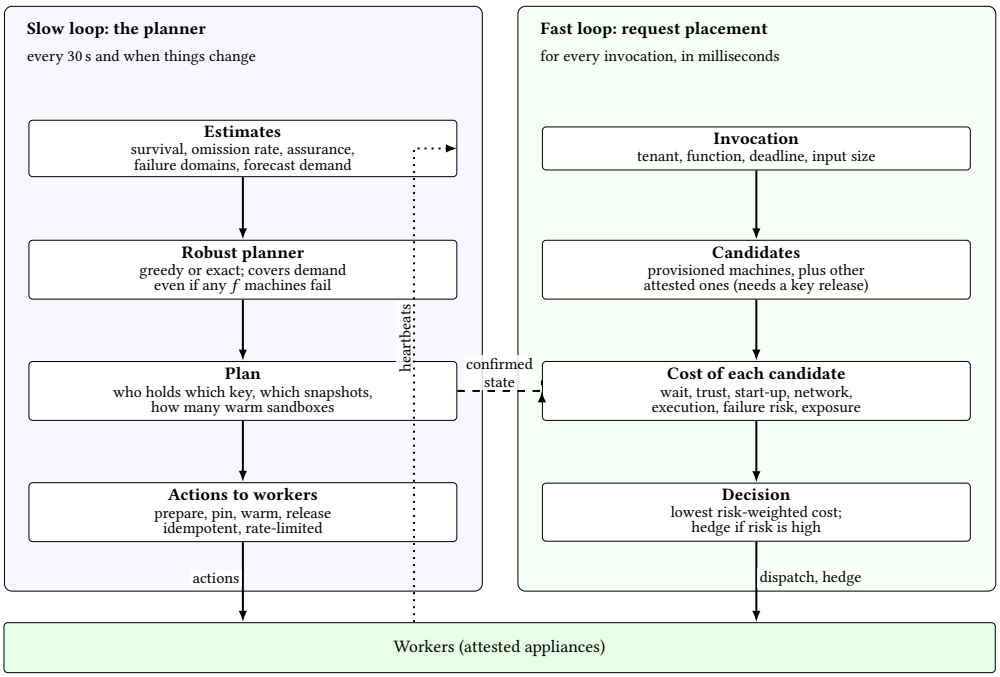


Fig. 4. The two parts of a scheduler. The planner decides where keys and warm sandboxes are kept and sends actions to workers. Request placement picks a worker for each request. Workers confirm completed plan actions in their reports, so placement never counts on a worker that is not ready yet.

5.2 What each part optimises

For a synchronous request the deadline is five times the function's run time on a warm sandbox, plus 50 ms. A request *misses* if it fails or finishes after its deadline. We report misses in every experiment. The two parts treat them differently.

Request placement minimises the chance that this request misses. For request i and candidate worker j it computes

$$J_{ij} = w_i \, P_{ij}^{\text{miss}} + \varepsilon \, \mathbb{E}[T_{ij}] + \lambda \, x_{ij} \tag{2}$$

and picks the smallest. P_{ij}^{miss} is the probability that the request misses on j: either j fails before finishing, or the predicted completion time exceeds the time left before the deadline. $\mathbb{E}[T_{ij}]$ is the expected completion time, which separates candidates with similar miss probabilities. x_{ij} is the exposure added if j does not hold the tenant's key yet: the tenant's sensitivity times $(1 - a_j)$. The weights are chosen so the terms are comparable: with $w_i = 1$ for synchronous requests, $\varepsilon = 0.05$ per second and $\lambda = 1$, one second of expected latency counts as much as a 5-point rise in miss probability, and giving a medium-sensitivity tenant's key to a new machine with trust score 0.8 counts as much as a 20% chance of missing. We minimise each request's own miss probability rather than a 99th percentile because a percentile describes many requests together and cannot be assigned to one decision; the average of the per-request miss probabilities is the miss rate.

The planner minimises cost while keeping enough capacity ready. When the planner runs, the requests it plans for do not exist yet; it has only a forecast. Minimising misses directly would also be circular, since misses depend on later placement decisions, which depend on the plan.

And warming every function on every worker would minimise misses while handing every key to every machine. So the planner instead requires enough ready capacity and minimises what that capacity costs.

Capacity is counted in slots. If a tenant's requests arrive at random times (a Poisson process) at rate λ_t and each occupies a slot for s_t seconds on average, the number running at once follows a Poisson distribution with mean $\lambda_t s_t$, whatever the distribution of run times. The planner provisions q_t slots, the 95th percentile of that distribution, so that in 95% of moments an arriving request finds a provisioned slot free. A request that finds none goes to a worker without the key and waits for a key release and a start-up. Real arrivals are burstier than Poisson, so during bursts fewer than 95% find a slot; the experiments measure the resulting misses directly.

The planner also tolerates the loss of any f machines. Let c_{tj} be the slots of worker j reserved for tenant t, and e_j the estimated probability that j stays in its session for the next 60 s and does not drop work. The plan must satisfy

$$\sum_j e_j \, c_{tj} \;-\; \big(\text{sum of the } f \text{ largest } c_{tj}\big) \;+\; u_t \;\geq\; q_t. \tag{3}$$

In words: after removing the f workers that contribute most, the expected remaining slots still cover q_t. The shortfall u_t is allowed but charged a high price, so it appears only when memory or trust limits make full cover impossible. Removing the largest contributions is the worst case, so no model of an adversary is needed. Bertsimas and Sim show how to write this "f largest" term as linear constraints [11], which lets an integer-program solver handle it. By default f is a tenth of the cluster, rounded up.

Subject to this, the planner minimises the memory and disk the plan uses, its key exposure, and the number of new key releases compared with the previous plan, minus the start-up time that warm sandboxes save. It also respects each worker's slots, memory and disk, caps how much of a tenant's capacity may sit in one city or one internet provider, and skips workers below the tenant's minimum trust score or whose certificate would expire within the next 60 s.

5.3 What the scheduler knows

Every number the scheduler uses is one of four kinds. *Reported* numbers come from workers every 2 s: busy slots, cached state, which keys a worker holds, when its certificate expires, its trust score. The scheduler adds its own assignments made since the last report. *Learned* numbers are estimated from past outcomes by the four estimators of Section 5.6. *Forecasts*, such as next minute's request rate, are computed from learned numbers. *Fixed* numbers are set before the run: start-up costs from the calibration, network delays from distance, each machine's declared speed and link rate, and the weights and tenant policies. The scheduler never sees when a session will really end, how long a request will really run, how a machine will really behave, or future requests; the code forbids the scheduling policies from importing the simulated world. The fixed start-up and network costs are the weak point: a deployment would measure them as it runs, and our prototype does not (Section 10.1).

5.4 The planner

The planner considers the live workers in its view that are not leaving (at most 40) and the 20 tenants and 50 functions with the highest forecast rate. Less frequent tenants and functions are served by request placement alone, on whichever worker it finds best. The evaluation workload has six tenants and 12 functions, so these limits never apply there.

We implemented two planners with the same inputs and outputs. The *exact planner* solves the problem as a mixed-integer program with about a thousand variables per cluster, using HiGHS [32]

with a time limit. The *greedy planner* works in two passes. First, it takes tenants in order of how much they would lose to cold starts (forecast rate times cold-start delay); for each tenant it adds workers one at a time, choosing the worker with the best ratio of reliability times trust score to memory and exposure cost, until the condition (3) holds. Second, on the workers that hold each tenant's key, it adds warm sandboxes and snapshots in order of start-up time saved per megabyte of memory, until each worker's memory or disk is used. The greedy planner takes about 8 ms per run. In a pilot, the exact planner needed 0.3–1.1 s for 10 workers, 4.7–5.0 s for 40 and 17–24 s for 80. Before running the experiments we fixed a rule: if the exact planner lowers 99th-percentile latency by less than 5% in every setting, the greedy planner is the method and the exact planner a reference (Section 9.3).

A new plan is computed only if the inputs changed beyond a tolerance; otherwise the old one is kept. The difference between the new and old plan becomes actions sent to workers: fetch the key and decrypt the bundle, keep a snapshot, start a warm sandbox, release. Actions are safe to repeat and limited in rate. Workers report completed actions, and the plan counts only what they have confirmed.

5.5 Request placement

For each request, placement sees only its metadata (function, tenant, time left before the deadline, input size, whether it is idempotent), the cluster view and the plan. It considers 12 candidates: workers that hold the tenant's key, and other attested workers, which would first need a key release and usually a start-up. Workers that are leaving, whose certificate is about to expire, whose trust score is below the tenant's minimum or that lack memory are excluded.

For each candidate it adds up five predicted delays and logs each: waiting for a free slot, getting the key (zero if held), starting the function at its current level, moving data over the network, and running. Completion time is modelled as a shifted log-normal distribution matched to these parts, and the failure probability is the chance that the worker ends its session or drops the work before finishing. A synchronous request goes to the lowest J at once. Asynchronous and background requests are collected for 0.2 s and assigned together by a minimum-cost matching. All baselines use the same candidates and cost model, so differences come from the decision rule alone.

If every candidate is almost certain to miss and even the shortest wait exceeds the time left, the cluster is overloaded: a synchronous request is rejected and an asynchronous one deferred. Requests that would miss only because of a cold start are accepted, because they warm the sandboxes later requests need. If the best candidate's miss probability is above 0.2, the function is idempotent and the cluster has used less than 10% of placements for duplicates, a second copy goes to the best candidate in a different city or internet provider, and the slower copy is cancelled when the first result arrives [22]. A request lost to a failure is placed again, excluding the failed worker, within a limit per request and per cluster.

5.6 Estimators

Four estimators turn past outcomes into the probabilities and times used above. For each we say what it predicts, where it is used, and why we chose it.

Session survival predicts the probability $S_j(\tau)$ that worker j's session lasts another τ seconds; the planner uses it for e_j and placement for the failure probability. We fit a Weibull distribution for each class of machine by maximum likelihood, counting sessions still running as censored observations, and refit every 60 s. Availability studies of volunteer machines find heavy-tailed session lengths that Weibull distributions fit well [34]. A Weibull with shape below one has a falling hazard, which captures what those studies show: a machine that has been up for an hour is less likely to leave in the next minute than one that just arrived. It has two parameters, so it can be fitted from a

few dozen sessions, and its conditional survival has a closed form. An empirical (Kaplan–Meier) estimate needs many more finished sessions and says nothing beyond the longest one observed. Until 20 sessions of a class have ended we use a prior (shape 0.7, scale 2 h). An owner's notice that they will leave can only shorten the predicted session, since notices may be false.

Dropped work predicts the probability p_j that worker j accepts a request and never returns a result. Each accepted request either returns or not, so we use a Beta distribution over that probability, updated by counting returns and losses during sessions. The update is two counters, and the prior (mean 2.4%) keeps a new worker from being judged on one or two outcomes, which a plain frequency would do. To catch a worker that behaves well and then starts dropping work, a CUSUM change detector [43] watches the same outcomes and resets the worker's estimate when the loss rate jumps; it reacts faster than a sliding window for the same rate of false alarms.

Run time predicts how long function f will run on worker j for a given input size. Run time grows roughly as a power of input size and scales with the machine's speed, so its logarithm is close to linear in the logarithm of input size, and we fit a linear regression in these logarithms. The regression is Bayesian so that it works from a handful of samples: the prior comes from the function's benchmark profile, and a per-worker correction is pulled towards zero until that worker has enough samples of its own. A forgetting factor lets it follow slow drift. Because the miss probability depends on the tail of the run-time distribution, the spread is recalibrated against the last 500 prediction errors (conformal calibration [47]). Gaussian processes or Bayesian additive regression trees would capture non-linear effects, but they need more data and more computation per decision than a millisecond budget allows, and we saw no non-linearity that would justify them.

Request rate forecasts each tenant's and function's arrivals for the next minute, for the planner. We count arrivals in 10 s bins and keep two exponentially weighted averages, one with a time constant of 60 s that follows bursts and one of 15 min that keeps the baseline, and take the larger. Taking the larger errs towards preparing too much, which is cheaper than preparing too little: an idle warm sandbox costs memory, while a missing one costs a cold start. Seasonal forecasters such as Holt–Winters need days of history that a new tenant does not have.

Each estimator keeps a few numbers rather than a table of past runs. They live in the scheduler's memory and are copied to the provider every 30 s, so a replacement scheduler can continue from them (Section 7.2).

5.7 Clusters and routing

In the clustered architecture the orchestrator groups workers by round-trip time into clusters of about 30 (between 10 and 60), each spanning enough cities and internet providers for condition (3) to be satisfiable, and moves workers between clusters only with hysteresis, since a moved worker loses its keys. A router scores each cluster for a request by the delay it expects there, in seconds: the round trip, the cluster's scheduling time and queue, a penalty if its recent miss rate is high, the key-release time if no worker in it holds the tenant's key, and the cold-start time if it has no warm sandbox of the function. Sending a request to another cluster is therefore a decision with an explicit price.

6 REACHING A SCHEDULER ON A VOLUNTEER MACHINE

In the clustered architecture each cluster's scheduler runs on one of its volunteer machines. Routers must send it requests, it must send assignments and plan actions to its workers, and workers must send it their reports. This section explains why ordinary request–response communication does not work here and what we use instead. In the global architecture none of this is needed: the scheduler runs on provider hardware at a stable address, and every worker simply opens a connection to it.

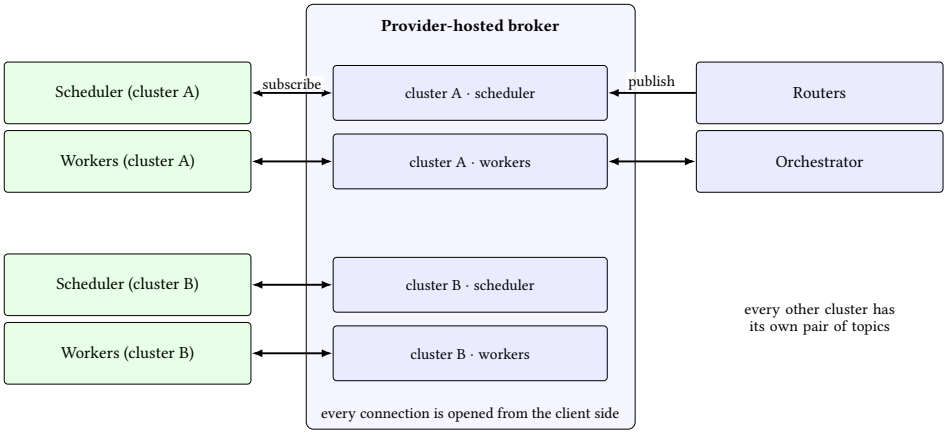


Fig. 5. Control messages go through topics on a provider-run broker, one set per cluster. Every connection is opened from the volunteer side. Request inputs and results do not use the broker: each worker keeps its own connection to its regional router.

6.1 The problem: volunteer addresses

Volunteer machines sit behind home or office routers that perform network address translation. Such a router lets a machine open connections to the outside, but it drops connections that arrive from outside, because it does not know which machine inside they are for. The public address of a home connection also changes when the router restarts or the internet provider reassigns it. A scheduler on a volunteer machine therefore cannot accept incoming connections, and its address lasts at most one session.

HTTP and gRPC both assume a server that listens at an address its clients know. Here the server would be a home machine with no way in and an address that changes, and every scheduler replacement would force every router and worker to learn a new address and reconnect at the same moment. Hole punching, where both sides connect outwards at once, works only for some pairs of home routers, and the rest need a relay anyway. With hundreds or thousands of clusters, each scheduler changing machines over time, neither approach scales.

6.2 Our design: a publish/subscribe broker run by the provider

The provider runs a *broker*: a public service at a stable address. Every party opens one long-lived connection to it. Messages are not addressed to machines; they are published to named *topics*, and the broker delivers each message to whoever has subscribed to that topic (Figure 5). Each cluster has its own topics, named by the cluster's number: the scheduler subscribes to the cluster's scheduler topic, each worker to its own topic and to a cluster-wide one, and routers publish requests to the scheduler topic. Nobody needs another party's address; they only need the cluster number, which the orchestrator gives them.

We take MQTT [41] as the protocol. It was built for devices on unreliable networks, its clients always connect outwards, and its at-least-once delivery suits our messages: plan actions are safe to repeat, and assignments carry the scheduler's epoch (Section 7.2), so a duplicate is recognised. Any broker with topics and outward connections would serve.

Replacing a scheduler becomes simple. The new scheduler subscribes to the same topic. The topic name does not change, so no address has to be announced and no worker has to reconnect.

Table 3. Where the scheduler runs. The volunteer-hosted design is the one whose failure handling we evaluate; E05 compares clusters with one global scheduler.

	Volunteer, per cluster	Provider, per cluster or region	Provider, global
How workers reach it	through the broker, by cluster topic	connect to its address	connect to its address
Extra provider service	broker	scheduler servers	scheduler servers
When the scheduler fails	epochs and a successor	provider replication	provider replication
Malicious scheduler host	possible	no	no
Extra hop per request	yes	no	no
Scheduling domain	tens of machines	any size	all machines

Workers only learn the new epoch and the new scheduler's certificate from the appointment message.

The broker cannot forge messages. MQTT alone does not protect messages from the broker that relays them, and Buccafurri et al. found no earlier proposal that gave end-to-end integrity against an untrusted MQTT broker [13]. We therefore add two layers. Each connection to the broker is mutual TLS with the party's certificate. In addition, every control message is signed end to end with the sender's session key and carries the epoch and a sequence number; the receiver checks the signature against the sender's certificate. A worker accepts an assignment only if it is signed by the scheduler the orchestrator appointed for the current epoch. The broker sees topic names, sizes and timing, and can drop, delay or reorder messages. These are omission and timing faults, which the detectors of Section 7 already handle. It cannot create, alter or replay a message.

Costs. A message from a worker to its scheduler now travels through the broker, one extra hop; a broker in the provider region nearest to the cluster keeps it short. The broker is also one more provider service that must stay up, and production brokers are run as replicated clusters for that reason. The traffic is small: about 0.46 KB/s per worker in a scalability pilot, or about 14 KB/s for a cluster of 30.

6.3 Why the scheduler is on a volunteer machine at all

The clustered design puts the scheduler on a volunteer machine so that the provider runs no server per cluster, and because it is the harder case for fault tolerance, so Section 7 tests the mechanisms where they matter.

Compute is not the reason. With the planning times we measured, running every cluster's scheduler on provider hardware would need about 0.9 processor cores for greedy planning at 10^5 workers in clusters of 30, and about one core for request placement at 1 000 requests per second. Only the exact planner would be expensive, at about 450 cores, and it was not measurably better (Section 9.3). Hosting on the provider would also remove one round trip from every synchronous request, remove a malicious scheduler host from the threat model, take the scheduler code out of the image, and allow larger scheduling domains. Table 3 compares the options.

6.4 What the simulator models

The simulator models network delay only: each message takes a time derived from the distance between cities, plus 20 ms for each end that sits behind a relay (15% of workers by default), which

stands in for the broker hop. It does not model the broker's capacity, connection set-up, address changes, reconnection, or the cost of signing messages. Sending every control message through a broker would add the hop for all workers, not 15%, and we did not sweep this. In E05 the global scheduler is also hosted on a volunteer machine, so it pays volunteer delays and failures that provider hardware would not; the comparison is therefore tilted towards the clustered design.

7 DETECTING FAILURES AND REPLACING SCHEDULERS

By Property 1, the same mechanisms must handle crashes, churn and malicious owners. This section says how a failure is noticed, how a failed scheduler is replaced, and how work is recorded.

7.1 Judging machines by returned work

A heartbeat is a periodic message in which a machine says it is running. It shows only that the machine can still send messages. The failures that matter most here keep heartbeats going: a worker that accepts requests and silently discards them (a *black hole*), a worker that behaves well for hours and then starts dropping work, and an owner who announces a departure that never happens. We therefore judge a machine by *returned work*: whether work it accepted comes back, as observed by a party that cannot be faked. Routers run on provider hardware, so their observations are trusted. Workers and schedulers run attested code, so their observations are honest, although their hosts can still stop or delay them.

Each detector watches for something that must happen if a component works. The router's *dispatch timer* expects the scheduler to acknowledge a request within 1 s; if it does not, the router suspects the scheduler. The scheduler's *result timer* expects a result by the predicted 90th-percentile completion time times 1.2, and at least 3 s; if none arrives, it probes the worker and suspects it. Three dispatch timeouts within 5 s make the router report the cluster as stalled to the orchestrator. The orchestrator also tracks how long the scheduler takes to place requests, from the scheduler's signed timestamps, and treats more than 200 ms as a failure. Finally, the dropped-work estimator and its change detector (Section 5.6) catch workers that drop work selectively or only after a while. Heartbeats remain as a fast way to notice plain crashes, but a working heartbeat never clears a suspicion. A suspected worker gets no new work, its unfinished work is duplicated or retried, and it is removed from the plan. A suspected scheduler is replaced.

7.2 Replacing a scheduler without an agreement protocol

The textbook way to replace a failed coordinator is to replicate its state across several machines with a consensus protocol. Among volunteers that would be slow, and it would bring back an agreement problem among machines that may be adversarial. Voss avoids it for two reasons: the provider's orchestrator is a reliable referee, and a scheduler keeps no state that cannot be rebuilt. Figure 6 shows the sequence.

Appointment and epochs. Each cluster has an *epoch* number in the orchestrator's database. The orchestrator appoints a scheduler by an atomic compare-and-swap that increments the epoch, much like a lease [14]. The appointment names the cluster, the epoch and the appointee's session certificate. Workers accept assignments only from the current epoch and the appointed certificate, and answer older epochs with a refusal, so an old scheduler that is still running is shut out. A scheduler that cannot reach the orchestrator for one lease period (5 s) also stops on its own. A cut-off scheduler can therefore cause only duplicate work, which is safe for idempotent functions; for a non-idempotent function a duplicate sent within the lease period could repeat its effect, and the protocol does not prevent that (Section 10.1).

Rebuilding the state. Requests not yet assigned are held by routers, which resend them. Running work and completed plan actions are held by workers. The estimators' numbers are copied to the

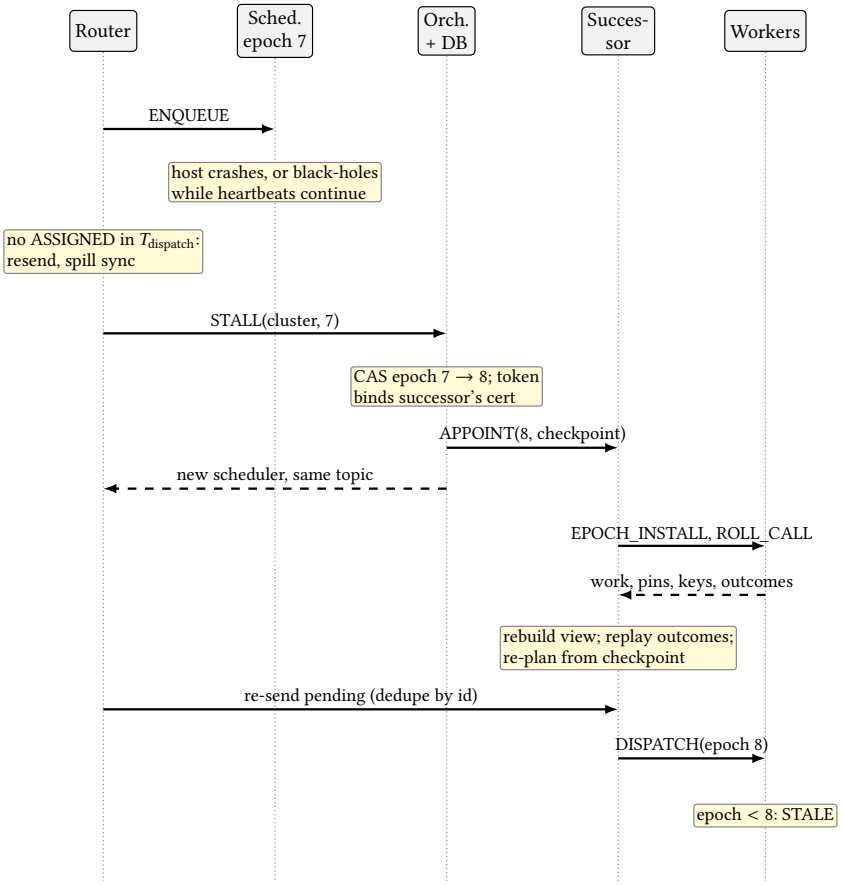


Fig. 6. Scheduler replacement. The router's dispatch timer detects a crashed scheduler or one that discards requests. The orchestrator increments the epoch atomically and appoints a successor, which rebuilds its state from the workers and the last copy of its estimators.

provider every 30 s. Work records are receipts held by routers. So the new scheduler installs its epoch, asks every worker for its running work, keys and cached state, restores the estimators from the last copy, and plans again. Because nothing authoritative lives on the old host, a scheduler machine that restarts is a new instance and must be appointed again.

Choosing a successor. The orchestrator keeps a ranked list of candidates by expected session length, speed, network position and trust score, spread across cities and providers. Any attested worker can serve, because the scheduler code is in the image, so promotion needs no boot. A planned handover, for example when the scheduler's owner announces a departure, first copies the estimators; a false announcement by a malicious owner only causes a clean handover. With the broker, the successor takes over by subscribing to the cluster's scheduler topic.

Worker failures. When a session ends, its keys and warm sandboxes are lost and its unfinished requests are retried or duplicated elsewhere, preferably on workers that hold the key. Per-cluster retry limits and spilling to other clusters prevent retry storms. An owner's notice of departure makes the scheduler stop sending new work to that machine, but never excuses work that machine dropped.

7.3 Receipts and credit

Each result carries a receipt signed with the worker's session key. It names the request and attempt, the worker and its certificate, the epoch, the run time, the amount of work in benchmark units, and a hash of the output. The router checks it against the certificate and the verifier's revocation list and stores it. A worker cannot claim work it did not do; it can only decline work, which the detectors above handle.

Receipts pay donors in *credit*, proportional to the benchmark work done rather than elapsed time, so a throttled machine earns nothing extra. Separately, the provider keeps a long-term *reliability score* per machine: whether its owner kept the commitments they declared, for example a promised all-night session that ended early. It is used only to reward donors and to break ties in the planner. It is kept apart from the session-survival and dropped-work estimators, which use other evidence, so no outcome is counted twice. We do not use a blockchain: a receipt signed by an attested key can already be checked by anyone, and the remaining risk, the provider deleting records, is better addressed by a signed append-only log.

8 HOW WE EVALUATE

A volunteer platform with thousands of machines, controlled churn, hostile owners and paired comparisons of policies cannot be run on real hardware. We evaluate Voss in a deterministic simulator and calibrate it against measurements. This section says what the simulator models, where its numbers come from, how far we checked it, and how every comparison is made.

8.1 The simulator

The simulator is a single-threaded discrete-event simulator in Python. Simulated time jumps from one event to the next in a fixed order, and every random draw comes from a named, seeded stream, so a run can be repeated exactly. A run takes a configuration, a calibration bundle and a seed. It first draws everything the system under test cannot influence: the machines, every session start and end, which owners are adversaries, the arrivals and the noise in each execution. These traces are content-hashed, so two systems compared on one seed face the same world. The engine then runs the actors (workers, schedulers, routers, orchestrator, verifier, key services) and the policies. Policies see only immutable views built from messages, never the world itself. At the end, checks assert that every success ran on an attested host after a key release, that each invocation has one outcome, and that the time components add up to the measured latency. Analysis reads only the raw tables.

Nothing is free. Every interaction is a message that arrives after a delay from the distance between about 100 cities (100 km per millisecond, times 1.3 for route detours), plus last-mile delay and jitter. Each worker has a download and an upload speed, drawn from a log-normal distribution with medians of 100 and about 30 Mbit/s, and all transfers to or from a worker share them. Disk reads, CPU contention, TPM quotes, key releases and the decision time of the planner, request placement, router and orchestrator all consume simulated time. Decision time is a fixed part plus a part proportional to the work, so results do not depend on the speed of the machine running the simulation. The coefficients are placeholders from the literature, not measurements. For the planner the charge is about 0.1 s, for the greedy and the exact planner alike, while the exact planner really takes 4.1 s, so its results are optimistic about how stale its plans are.

The world. Workers come in three tiers (nightly, idle-hours, dedicated) with different hardware, cities and trust technologies. Session lengths are heavy-tailed with time-of-day effects, most departures are abrupt, and correlated shocks remove a share of a city at once. No public trace of consumer donation sessions exists, so we follow volunteer-availability studies [34], mark the

parameters *assumed* and sweep them. The workload has six tenants and 12 functions modelled on SeBS [18], in three service classes, with skewed popularity and bursty, day-and-night arrivals [48]. 11 of the 12 functions are marked idempotent, by our assumption. A fraction α of workers are adversaries. They drop some or all accepted work, delay it, behave well and then start dropping work, announce departures that never happen, or act together within one city or internet provider. By Property 1 none of them forges results or reads keys on an attested machine; without attestation, both are possible.

8.2 Calibration and what we checked

Parameters from the specification are literature values or assumptions and are tagged so. Measured values come only from calibration bundles. We measured one on a consumer laptop (Intel Core i3-7020U, a discrete TPM 2.0, a hard disk), which stands in for a donor machine. A harness reboots it into the image and records timestamps at about one-second resolution, and microbenchmarks cover TPM quotes, hashing, microVM boot and snapshots. Compared with our placeholders, reboot to kernel is slower (21 s against 15 s), the phase from kernel start to a running agent is far slower (31 s), and creating a 512 MB snapshot on a hard disk takes 6.7 s. From power-on, an attested worker takes 65 s (median of 9 boots, 44 s of it after kernel start). A warm-cache microVM boot takes 154 ms, close to published figures [2].

Two limits apply. The laptop is not the intended donor machine and lacks AMD SEV-SNP and Intel TDX, so confidential-VM parameters are literature values, which we sweep [30, 37, 39]. And we ran the image's measurement and quote steps on the laptop's installed operating system, with microVMs through Firecracker, rather than booting the final locked image on bare metal. The time to an attested worker is therefore a sum of measured steps, not one measured boot of the final image.

A first check against real execution (E01). To test the start-up and execution parts of the simulator, we ran the same scenarios in a single-box emulation on a lab node (Intel Core i7-12700, 125 GB, NVMe, Firecracker 1.10.1) and in the simulator, with a calibration measured on that node. The emulation uses real microVMs for boot, snapshot restore, dispatch and function execution, and keeps network, key release, attestation and queueing as models (16 donor slices, five functions, ten minutes per run, three seeds). Success rate agrees within 0.6 percentage points in every scenario, the mix of start-up levels is identical, and median synchronous latency agrees within 6–23%. The *shape* of the latency distribution does not meet our pre-set cut-offs, so every scenario is graded "partly" close and none "close". At light load the simulator's 99th-percentile latency is 3.7 to 4.1 times the emulation's, driven by a handful of cold invocations: the P99 of about 530 requests is the five slowest. E01 does not test the network, attestation, key release, session boot or failure detection at scale, and no multi-machine deployment has validated the simulator end to end. We therefore read differences between designs, not absolute latencies.

8.3 Protocol

Each experiment is a frozen specification, and every cell is run with ten seeds (three for the two largest scalability points). Unless stated, Voss uses the greedy planner and the default request placement, routing and duplication rules. Systems in a comparison share the world and the per-invocation random draws, so each comparison is *paired* by seed. For a metric we take the per-seed difference and report its mean with a 95% bootstrap interval [24]. Percentiles are computed per run, then averaged, never pooled.

Each hypothesis is written down before the run with a mechanical rule. It is *supported* if the interval excludes zero in the predicted direction by at least a stated margin, *weak* if it excludes zero but the effect is below the margin, *inconclusive* if the interval includes zero, and *refuted* if it

excludes zero the other way. The manuscript's numbers and tables are generated from the result tables by a script that also fails if a result stops supporting the sentence it backs.

We report the share of requests that succeed, the share that meet their deadline, the median and the 99th percentile of synchronous latency (written P99 from here on), extra attempts per successful request (from retries and duplicates), memory held by idle warm sandboxes, and key exposure. We call a system *feasible* in a setting if the lower 95% bound of its success share is at least 0.99 and either its share of deadlines met or its P99 meets a target. Maps of feasibility then show *where* each design works, rather than an average over settings.

Baselines run on the same simulator, candidates and cost model and differ only in one policy: no planner (a worker gets a tenant's key only when a request needs it), a keep-alive timer (a sandbox stays warm for 10 minutes after use), heartbeat-only failure detection, and $f = 0$ (no protection against losing machines). The *simple baseline* combines static regional clusters, nearest-region routing, least-loaded placement, keep-alive and no hedging. Commercial-style references (*Lambda-like*, *Azure-like*) are always-on datacenter hosts with per-function keep-alive and nearest-region routing, priced from list prices, on the same engine.

8.4 Reproducibility

Every number, table and figure in Section 9 is regenerated from raw run logs by committed scripts, and runs are bit-reproducible: re-executing a run gives raw tables with the same result hash, recorded in a manifest with the scenario and calibration hashes. Floating-point arithmetic repeats bit for bit only under stated conditions, because a last-bit difference can flip a near-tie in placement and send the whole trajectory elsewhere. We found that NumPy's maths functions use different code paths on CPUs with and without AVX-512, and a first verification re-run failed for exactly this reason. We now pin NumPy to the SSE4 baseline that every x86-64 CPU supports and record a canary hash of the maths functions in each manifest. The full sweep, 21 experiments and 2,366 runs, was then re-executed on independently provisioned machines, and 2,366 of 2,366 hashes matched. Reproduction needs an x86-64 CPU with fused multiply-add, Linux with glibc, Python 3.11 and the committed lockfile, and about four hours of wall-clock time as parallel jobs. No TPM or special hardware is needed. Appendix A gives the requirements, checks and commands.

9 RESULTS

We organise the results around the questions the design raises. Appendix B lists every hypothesis with its verdict.

9.1 What does trust cost, and how does it shape start-up?

The TPM image pays a large one-off cost for each session, 19 s with our pre-measurement parameters and 65 s from power-on on the calibration machine. A confidential VM needs 1.4 s and a confidential-container pool 8.1 s. We had expected confidential VMs to be 10 to 100 times *slower* at establishing trust. That was refuted: they are about 93% faster, because the TPM image's cost is a reboot. Once the session exists, though, the TPM image is as fast as an unattested Firecracker host: warm and snapshot starts are within a few percent of the unattested figures (H5, supported), because attestation is paid per session, not per invocation.

9.2 Where does volunteer serverless work at all?

Averages hide the regimes in which a design fails, so we map feasibility. Figure 7 colours each cell by which of Voss and the best simple baseline is feasible (success at least 0.99 and 95% of deadlines met), and prints Voss's P99.

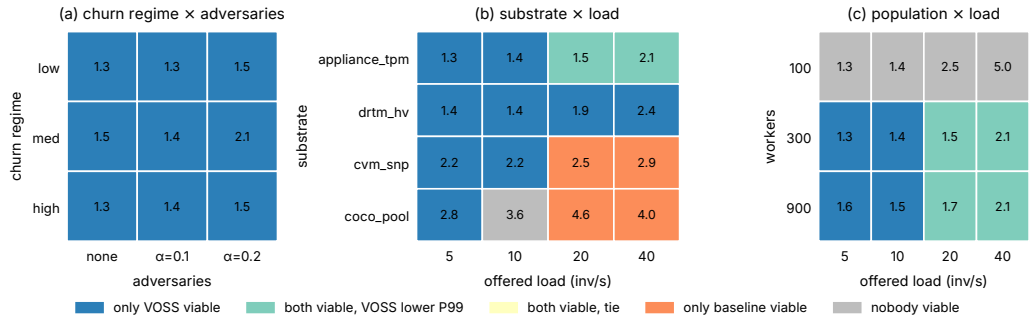


Fig. 7. Feasibility maps. Cell colour shows which of Voss and the best simple baseline meets the target (success at least 0.99 and 95% of deadlines met). Numbers are Voss's synchronous P99 in seconds. (a) churn against the fraction of owners who drop work, (b) trust technology against load in invocations per second, (c) population against load.

Table 4. Planner comparison (E04): mean ± 95% half-width over 10 seeds. Met: share of deadlines met; Warm: GB of idle warm memory.

Regime	Planner	P99 (s)	Met	Warm GB	Exposure	Solve (s)
churn_med	no planning	2.78±1.35	0.940±0.017	1±0	8.5±1.4	0.002±0.000
churn_med	keep-alive TTL	1.78±0.48	0.971±0.018	61±9	7.4±1.1	0.002±0.001
churn_med	greedy planner	1.29±0.09	0.978±0.015	94±14	19.9±1.8	0.008±0.001
churn_med	robust MILP	1.27±0.10	0.979±0.015	129±18	20.1±1.5	4.005±0.505
churn_high	no planning	3.48±1.32	0.923±0.029	1±0	7.5±1.2	0.001±0.000
churn_high	keep-alive TTL	3.28±1.91	0.940±0.030	59±10	7.3±0.9	0.001±0.000
churn_high	greedy planner	1.42±0.17	0.958±0.028	87±13	15.8±1.2	0.008±0.002
churn_high	robust MILP	1.31±0.09	0.958±0.031	119±11	15.2±0.9	4.164±0.650

Voss is the only feasible system in all 9 cells of panel A, which spans three churn levels and up to 20% of owners dropping work, in 9 of 16 cells of panel B and in 4 of 12 cells of panel C. With an absolute P99 target of 3 s it is feasible in 9 of 9, 13 of 16 and 10 of 12 cells, and the simple baseline in none. At 5 s, Voss is feasible in 9, 16 and 10 cells.

The map also shows where Voss is the wrong choice. On confidential virtual machines at higher load, the simple baseline is the only feasible system in 4 cells. We conjecture that with cheap session set-up there is little cold-start cost left for planning to remove, but we did not test this. With 100 workers no system is feasible in any cell of panel C, so the design needs a minimum population, and at a deadline-met target of 0.98 no system is feasible anywhere.

9.3 Is the planner needed?

Section 5.1 argued that keys and warm sandboxes must be planned ahead. We test that by comparing planners with two single-loop alternatives on identical scenarios (Table 4): no planner, where a worker gets a tenant's key only when a request needs it, and a 10-minute keep-alive timer. The greedy planner lowers synchronous P99 by 54% at medium churn and 59% at high churn against no planner, and by 27% and 57% against a keep-alive timer. The exact planner gives 54% and 63% against no planner, and 29% and 60% against keep-alive (H1, supported). Planning costs donor memory: the exact planner holds about 2.1 times the warm memory of keep-alive at medium churn.

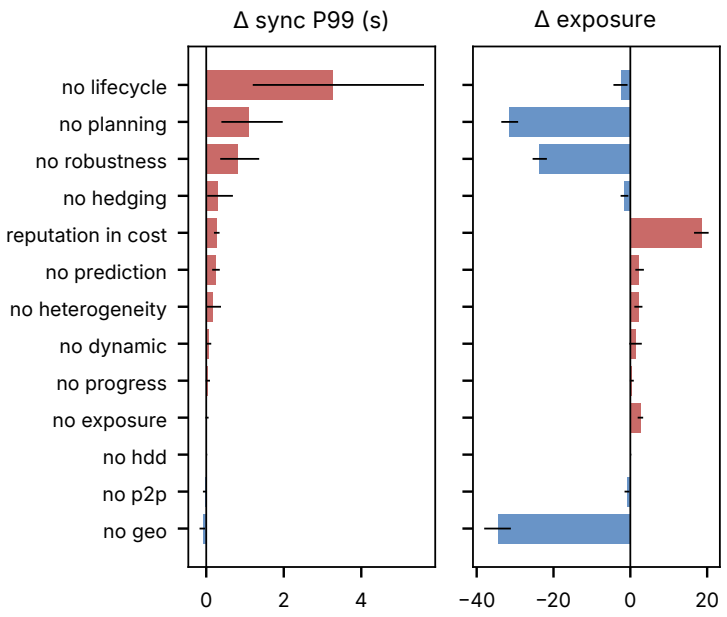


Fig. 8. Ablations (E19): change when one component is removed, relative to full Voss (paired, 95% intervals). Left: synchronous P99. Right: key exposure.

The extra gain from the exact solver is small and not established. At medium churn it improves P99 by 1.8% over the greedy planner, below our 5% threshold. At high churn its point estimate is 8.1%, above the threshold, but the 95% interval runs from a 21% reduction to a 1% increase and includes zero (H8, inconclusive). Ten seeds can neither confirm that the exact solver matters nor show that it does not. Its solve takes 4.1 s per cluster against 8 ms, yet the simulated clock charges both about 0.1 s. Our rule required the gain to be below 5% in every regime, and the data do not show that at high churn, so we do not claim the rule is met. We keep greedy as the default, a choice made before these runs.

Both alternatives are simple. A stronger single-loop rule, such as a keep-alive time learned per function from its idle times [48] combined with pre-warming on predicted arrivals, is not in this comparison, so these numbers show that planning beats simple reactive rules, not every reactive rule.

To see which components matter, Figure 8 removes one at a time (10% of owners dropping work). Making request placement ignore start-up levels costs 3.3 s of P99, removing planning 1.1 s, removing the protection against losing f machines 0.8 s and removing hedging 0.3 s. The components that buy P99, planning and robustness, also raise exposure: without planning, total exposure is lower by 31 units (Eq. 1), because fewer hosts hold keys. Removing the exposure term changes P99 by only 0.02 s and raises exposure by 2.6, so limiting exposure costs no measurable latency (Section 4.4). Geo-clustering is the only component whose removal *improves* P99 (by 0.09 s) and also cuts exposure (by 34 units). We return to this in Section 9.6.

Table 5. Scheduler faults (E15): time to detection and failover, success, and P99 inside the fault window.

Fault	Detector	Detect (s)	Failover (s)	Success (%)	P99 in window (s)
crash	progress	2.2±0.3	2.7±0.4	99.95±0.02	1.28±0.15
crash	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.66±0.33
partition	progress	2.2±0.3	2.6±0.4	99.95±0.02	1.28±0.16
partition	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.67±0.31
blackhole	progress	4.2±3.0	4.6±3.0	99.95±0.02	1.28±0.15
blackhole	heartbeat_only	60.0±0.0	59.3±1.1	96.81±0.51	2.90±0.08
slow	progress	13.9±7.9	14.3±8.2	99.95±0.03	1.34±0.14
slow	heartbeat_only	60.0±0.0	59.3±1.1	99.55±0.41	2.00±0.33

Table 6. Owners who drop work (E08): mean ± 95% half-width over 10 seeds. Omissions are results dropped by adversaries.

Adversaries	System	Success (%)	P99 (s)	Waste (%)	Omissions
none α=0	VOSS	99.99±0.01	1.27±0.11	1.7±0.5	0±0
none α=0	VOSS, f=0	99.99±0.01	1.60±0.24	2.2±0.6	0±0
none α=0	heartbeat-only	99.98±0.01	1.36±0.15	2.0±0.7	0±0
mixed α=0.1	VOSS	99.95±0.04	1.32±0.12	2.2±0.5	38±14
mixed α=0.1	VOSS, f=0	99.96±0.03	2.15±0.60	2.9±0.6	37±11
mixed α=0.1	heartbeat-only	98.19±0.75	1.36±0.13	4.0±0.9	298±115
mixed α=0.2	VOSS	99.95±0.03	1.43±0.16	2.6±0.5	61±13
mixed α=0.2	VOSS, f=0	99.83±0.17	2.05±0.52	3.5±0.8	80±31
mixed α=0.2	heartbeat-only	96.79±1.13	1.39±0.16	5.6±1.3	527±161
all α=0.2	VOSS	99.94±0.05	1.37±0.10	2.3±0.7	31±9
all α=0.2	VOSS, f=0	99.96±0.03	2.28±1.00	2.7±0.5	28±8
all α=0.2	heartbeat-only	97.70±1.26	1.42±0.14	5.2±2.4	385±227

9.4 Does progress-based detection make malice look like churn?

Scheduler faults. Table 5 injects crashes, partitions, black holes and slowdowns into scheduler hosts. Heartbeats alone cannot see a black hole, so detection waits for a 60 s timeout (60 s) and the run suffers 132 timeouts, against 4.2 s and 0.7 with progress-based detection (H-FT1, supported). For crashes, detection takes 2.2 s and failover 2.7 s. A slowed but live scheduler is the hardest case: failover takes 14 s because evidence must build up before a slowdown can be told from load.

Owners who drop work. Table 6 raises the fraction α of owners who drop work. At $\alpha = 0.2$, Voss keeps 99.95% of requests succeeding, while heartbeat-only detection with otherwise identical machinery falls to 96.8% (H4, supported) and loses 9 times more results to them. Success with these owners stays near the level with churn alone, which supports the claim that attested owners who drop work behave like churn once machines are judged by returned work. The robust-coverage constraint is not what keeps success. Setting $f = 0$ changes success by only 0.11 percentage points (H2, inconclusive), though it lengthens P99. Detection and re-dispatch carry success; robust planning buys tail latency.

Duplicates. Always sending a duplicate cuts P99 but wastes 10.6% extra attempts. Duplicating only requests whose miss probability exceeds 0.2 wastes 1.8%, and its P99 is 13% lower at $\alpha = 0.1$, with an interval from -32% to +1% (H-FT3, inconclusive). It is cheap insurance, not shown to be necessary.

9.5 What do robustness and exposure cost?

Robustness f and the exposure weight trade latency against key exposure (E23). Exposure grows with f (13, 19 and 24 at $f = 0$, 0.1 and 0.2), as expected, since covering any f failures needs more key-holding hosts. Aggregate P99 stays within 1.27–1.35 s across configurations, so the knee at $f = 0.1$ saves 25% of exposure against the best-latency point for 1.6% more P99 (H3, supported). Per tenant class the picture is less benign. Low-sensitivity tenants stay at about 1.4 s or below, but for high-sensitivity tenants P99 reaches 1.8 s in one robust configuration. The aggregate knee should not be applied to every tenant class, and each needs its own operating point.

9.6 Which components do not earn their complexity?

Geo-clustering. We compare one global scheduler with static regions, static k-means and dynamic clusters on 600 workers (about 225 of them attested) at 20 invocations per second, with clusters of about 20 to 35 attested hosts. The global scheduler plans over at most 40 hosts, which is part of why it holds fewer keys, and nothing in this load stresses one scheduler's capacity. Dynamic clusters (E05) have *higher* P99 than the global scheduler by 17% in Europe, 17% in Europe and North America and 5% worldwide (H7, refuted in all three), and hold 5.1 times the exposure in Europe, since each of the 9 clusters must cover demand with its own key holders. In this experiment the global scheduler was itself hosted on a volunteer machine; on provider hardware it would avoid volunteer delays and failures, so the comparison favours clusters. We attribute this to fewer hosts per cluster for pooling warm state and keys, but did not isolate the mechanism. We found no population in which geo-clustering wins at this scale.

Routing, caching, reliability score. Without flash crowds, nearest-region routing meets 0.991 of deadlines and least-loaded routing 0.911, because it ignores distance. Cost-aware caching with peer fetch and a disk tier is at most 3% better than plain LRU in P99, and the intervals include zero (H6, inconclusive); all variants start about 98% of invocations from a warm sandbox, which leaves little for disk tiers to improve. Receipts stay off the critical path (H10, supported): lengthening settlement from 10 to 300 s changes P99 by at most ±1.5%, and the stream is small, about 17 rows and 4.5 KB per second at 20 invocations per second. Using the reliability score to break ties adds little (2.4% lower P99, weak), and credit is highly concentrated (Gini 0.93; the top 10% of workers earn 93% of credit and 65% earn almost none), which is a fairness problem for an incentive that relies on credits.

9.7 How does Voss compare with baselines and a commercial reference?

Simple baseline. On a homogeneous, stable, benign pool we expected the simple baseline to match Voss (E10). It does not: Voss has 64% lower P99 (H11, refuted), which we attribute to planning's pre-warming against churn. The baseline holds about half of Voss's key exposure, though (Voss is 1.9 times higher). Whether the latency is worth the exposure is a policy choice that the trade-off front of the previous section lets a deployer make.

Commercial reference. On the same workload at list prices (E17), Voss's P99 at 20 invocations per second is 14% and its median 45% above the Lambda-like reference. Cost per million invocations, including the control plane and excluding volunteer credits, falls from 20 USD at 10 invocations per second to 10 at 20 and 5 at 40 as the fixed tier is amortised, against about 12 USD for the references. Voss is cost-competitive only above roughly 20 invocations per second on this workload and never has lower latency.

Scale. We grow the population from 10^2 to 10^5 workers at constant load per worker in a cheaper mode (coarse heartbeats, greedy planning, two simulated minutes, E18). Up to 10^4 workers success is 99.8%. At 3×10^4 it falls to 97% with a decision P99 of 14 s, and at 10^5 to 81% with a synchronous

P99 of 28 s and a decision P99 of 30 s. Decision latency grows from about 0.1 s to tens of seconds, while control traffic per worker stays flat (0.70–1.08 messages per second), so the limit lies in the central decision path and not in message volume. We did not isolate which component saturates first, and the two largest points use three seeds.

9.8 Do the conclusions depend on our calibration?

We repeated the key comparison under the placeholder and laptop-measured bundles (E24). Voss's P99 is 1.17 s with measured parameters against 1.32 s with placeholders, while the simple baseline moves from 3.9 s to 4.6 s. The advantage of planning therefore grows under measured parameters. Absolute values should not be read as predictions.

9.9 Summary of verdicts

Of the hypotheses stated before the runs, 11 are supported, 2 weak, 6 inconclusive and 5 refuted. The refuted ones (geo-clustering, the simple baseline on homogeneous hosts, the cost ratio of confidential VMs) and the inconclusive ones (the robust constraint for success, the exact solver, caching, hedging) are as informative as the supported ones.

10 DISCUSSION

What the evidence supports. The central claim holds in simulation: planning ahead which machines hold keys and warm sandboxes beats simple reactive rules. Planning cuts tail latency by roughly half under churn, lifecycle-aware placement and planning are the two largest contributors in the ablation, and progress-based detection is what keeps success high when owners omit. These statements concern mechanisms and are backed by paired intervals. They hold across churn levels and adversary fractions in the feasibility maps, not at one operating point.

What it does not. Four expected benefits did not appear. Geo-clustering is slower than a single scheduler and holds several times the exposure at our scale. The exact solver is not demonstrably better than the greedy planner, so its cost is justified only by the coverage guarantee, which we could not show matters for success. Cost-aware caching and the reliability score change P99 by a few percent at most, and credit is extremely concentrated. A commercial-style reference is faster, and Voss is cheaper per invocation only once its fixed control plane is amortised. None of this contradicts the premise that volunteer hardware can serve latency-sensitive functions. It narrows which components earn their complexity. A leaner Voss (attestation, a greedy robust planner, a cost-based placer with hedging, progress-based detection, epoch-fenced failover and receipts) is the version our evidence favours.

Exposure is the price of latency. Planning, robustness and warm state all enlarge the set of machines holding a tenant's key. The simple baseline has about half of Voss's exposure, and moderate robustness captures most of the benefit in aggregate but not for every tenant class. A deployment should offer exposure and robustness as per-tenant policy, not global constants. The feasibility maps add that the trust technology and a minimum population matter more than tuning: on confidential virtual machines the simple baseline can suffice at high load, and below a few hundred hosts no system is feasible.

Reliance on attestation. Every robustness result uses omission and timing faults only. If attestation failed, for example through the physical attacks we excluded, adversaries could forge results and read keys, and exposure would stop being a proxy for risk.

Where the scheduler runs. Section 6 argued that a volunteer-hosted scheduler needs a broker and a provider-hosted one does not. The planner and request-placement results do not depend on the scheduler's location, except through one router-to-scheduler round trip and decision latency. The scheduler-fault results are specific to volunteer hosting, and the geo-clustering result suggests

that larger scheduling domains, which a provider-hosted scheduler can offer, are better. We did not run the provider-hosted variant. We expect the planner and request-placement conclusions to carry over and the fault-tolerance results not to, but that is a prediction.

A better-founded planner. The greedy planner was chosen because it is cheap and the exact program did not beat it, but its scores are hand-built. In particular, it values the tenth warm sandbox of a function as highly as the first, although the first serves most requests and the tenth very few. Treating each tenant's prepared slots as an Erlang loss system, in which a request that finds every slot busy goes to the slower path, gives the missing structure: the chance of finding every slot busy depends only on the mean run time, and it falls with each added slot by less than the previous slot. Warm sandboxes can then be allocated by marginal analysis, which is optimal for such diminishing returns, and key holders chosen by greedy selection with approximation guarantees. We have designed this planner and leave its evaluation to future work.

Function workflows. We treat invocations as independent, but many applications are graphs of functions in which one completion triggers the next [17], and each stage can pay a cold start in turn [21]. A trust-aware scheduler could prepare the keys and warm state of a function's successors while it runs. The risk is speculation: warming a function that a branch never calls wastes memory and may widen exposure, so look-ahead warming should be budgeted like hedging. We do not evaluate this.

10.1 Limits of this study

- **Simulation, partly validated.** No multi-machine deployment has validated the simulator. The single-box emulation (E01) agrees on success and start-up levels but not on the shape of the latency distribution, and it did not exercise the network, attestation, key release, session boot or failover. Policies are the real code, while the world (network, churn, adversaries) is a model.
- **Calibration scope.** One consumer laptop with a hard disk, with the image's measurement steps run on the installed operating system. Confidential-VM and hypervisor parameters are literature values. The main comparison survives a change of bundle (E24), but another CPU, disk or firmware could shift absolute costs.
- **Assumed workload and churn.** No public trace of donation sessions exists. The workload has 12 functions of six tenants, so the planner's limits of 20 tenants and 50 functions never apply and the long tail of rarely used functions is not exercised. Functions make no calls to external services. 11 of 12 are marked idempotent by us, and the retry, hedging and success results under omission rest on that. This matches what providers already ask of developers (Section 2), but we did not measure how many real functions meet it. The adversary uses a fixed menu of strategies, and an adaptive one could do worse.
- **Idealised knowledge of costs.** The scheduler uses calibrated start-up, trust and network times and takes declared host speeds as exact, whereas the simulated world adds noise. We did not sweep prediction error.
- **Network and decision costs.** Network delay is geographic plus 20 ms for 15% of workers behind a relay. Broker capacity, connection set-up, address changes and reconnection are not modelled, and decision latency is a placeholder formula.
- **Scale and statistics.** The two largest scalability points use three seeds and a cheaper mode, and the geo-clustering comparison is too small to stress one scheduler. Ten seeds give wide intervals for tail metrics, and many experiments were run, so a single verdict should be read with that in mind. Literature parameters are quoted as reported and tagged for re-verification.

10.2 Future work

The most valuable next steps are to evaluate the loss-model planner described above against a stronger single-loop baseline; to validate the simulator on a small real testbed, which would also test the host-mode assumption; to compare the three scheduler placements of Table 3 under a load that stresses one scheduler, with a sweep of the relayed share; to measure confidential-VM hosts and refit decision costs; to revisit geo-clustering at larger populations; to evaluate workflow-aware warming; and to study adaptive adversaries and the concentration of credit.

11 RELATED WORK

Voss needs six things at once: tolerance of untrusted machines, tolerance of churn, latency targets, planning of keys and warm sandboxes, detection of gray failures, and verifiable records of work. Each line of work below covers some of them.

Volunteer computing. BOINC-style systems use idle machines for scientific batch work [6, 7]. Because the machines are untrusted, they run each task several times and accept a result when enough copies agree, and they use deadlines of hours to tolerate machines that disappear. Replication multiplies cost, needs tasks that always give the same answer, protects results but not the confidentiality of inputs, and works on a timescale of hours. Voss uses attestation for integrity and confidentiality instead, and duplicates work only against dropped results, and only for requests at risk.

Serverless scheduling and cold starts. Work on cold starts decides when to keep sandboxes alive or start them early [48], treats keep-alive as caching [27], or makes start-up faster with lighter sandboxes and snapshots [2, 8, 15, 40]. For placement, Sparrow schedules short tasks by sampling a few machines [42], Falcon moves a central scheduler into a programmable switch to cut its latency [35], and Borg packs long-running jobs onto large fleets [51]. Inside one machine, ALPS learns a priority schedule for function processes from recent traces [26]. All of these assume trusted machines that stay up: a warm sandbox disappears only when a policy evicts it, and there is nothing like a key whose release costs time and adds exposure. Voss reuses the start-up levels and the per-request placement ideas, and adds a trust step per session and a plan that accounts for churn and exposure.

Two-timescale placement in edge computing. Mobile edge computing commonly splits decisions by timescale: which services or resources to place on edge servers is decided slowly, and where each request goes is decided quickly. Farhadi et al. place services and schedule requests at two timescales with an approximation guarantee [25], and Tang et al. combine slow resource provisioning by Gibbs sampling with fast workload scheduling for cloud-assisted edge computing [49]. Voss shares this structure, and Section 5.1 gives our reason for it. What differs is the resource being placed: trust in the form of keys, whose cost is exposure and whose benefit is the slow path avoided; machines that disappear with probabilities we estimate; and protection against worst-case rather than random losses. The Lyapunov drift-plus-penalty method often used in that literature is a natural way to turn our exposure weight into a budget.

Harvested and edge serverless. Platforms on spare datacenter capacity [53] and edge schedulers [44] run on machines that belong to the provider or a trusted party, so they handle announced evictions, not machines that silently stop answering, and give no secret to a party that might read it.

Confidential serverless. Trusted execution environments protect code from the host operating system and the platform operator [5, 9, 19], and recent work runs serverless functions in confidential virtual machines and containers [1, 39, 46]. These designs target provider-managed servers. They do not deal with machines whose sessions end without warning, and they do not schedule the

establishment of trust as a resource with a cost, a lifetime and an exposure. Voss can run over either kind of machine.

Trusted components and failure detection. Small trusted components, such as a counter or log, restrict what a faulty node can do and reduce the replication needed for Byzantine agreement [36, 52]. We use attestation differently: it does not make agreement cheaper, it removes the need for it, because an attested machine can stop but not lie, so a provider-run referee can replace agreement among volunteers. For detection, heartbeat detectors, including the adaptive φ-accrual detector [29], estimate whether a process is running from the timing of its heartbeats. Gray failures [31] show why that is not enough, and Voss instead checks whether accepted work comes back.

Brokers and duplicates. MQTT [41] gives no protection against the broker itself, and MQTT-I adds end-to-end integrity for data flows through an untrusted broker [13]; we sign messages end to end for the same reason. Hedged requests reduce tail latency by duplicating slow work [22]; Voss duplicates in proportion to estimated risk, on a machine in a different city or provider, within a budget. For protection against losing f machines we use the budgeted robust optimisation of Bertsimas and Sim [11], which answers "is demand still covered if any f machines fail together?" without a probability model of the adversary.

12 CONCLUSION

Serving latency-sensitive functions from volunteer machines needs more than running tasks several times and keeping sandboxes alive. Churn destroys the state that makes short functions fast, and untrusted owners make that state risky to create. We argued that attesting a locked image reduces hostile owners to the stops and delays that churn already causes, and that what remains is a scheduling problem: deciding which machines are attested, given a tenant's key and kept warm, when all of that is slow to set up, lost at the end of every session, and a leak risk on every extra machine. Voss plans this state every 30 seconds with spare capacity for lost machines, places each request by its own chance of missing the deadline, judges machines by the work they return, and replaces failed schedulers through a provider-run coordinator without an agreement protocol among volunteers. In the clustered architecture, schedulers on volunteer machines are reached through a provider-run publish/subscribe broker; in the global architecture, the scheduler runs on provider hardware. In a calibrated simulator Voss keeps P99 under 3 s in 9 of 9 and 13 of 16 settings where simple baselines do not, lowers P99 by 54–59% against provisioning on demand, and keeps 99.95% of requests successful when 20% of owners drop work. The advantage is not universal: on confidential virtual machines at high load the simple baseline is the only feasible system, and geographic clusters, cost-aware caching and the reliability score did not pay for themselves. The next steps are the loss-model planner of Section 10 and a multi-machine testbed to validate the simulator and the networking design that this paper models only as delay.

REFERENCES

[1] 2025. Confidential Serverless Computing (Wallet). arXiv:2504.21518; USENIX NSDI 2026. TODO verify authors and final title.

[2] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proc. USENIX NSDI*. 419–434.

[3] Amazon Web Services. 2026. How Lambda handles errors and retries with asynchronous invocation. AWS Lambda Developer Guide, https://docs.aws.amazon.com/lambda/latest/dg/invocation-async-error-handling.html. TODO verify; accessed October 2026.

[4] Amazon Web Services. 2026. Understanding retry behavior in AWS Lambda. AWS Lambda Developer Guide, https://docs.aws.amazon.com/lambda/latest/dg/invocation-retries.html. TODO verify; accessed October 2026.

[5] AMD. 2020. SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. White paper.

[6] David P. Anderson. 2004. BOINC: A System for Public-Resource Computing and Storage. In *Proc. IEEE/ACM Int. Workshop on Grid Computing*. 4–10.

[7] David P. Anderson, Jeff Cobb, Eric Korpela, Matt Lebofsky, and Dan Werthimer. 2002. SETI@home: An Experiment in Public-Resource Computing. *Commun. ACM* 45, 11 (2002), 56–61.

[8] Lixiang Ao, George Porter, and Geoffrey M. Voelker. 2022. FaaSnap: FaaS Made Fast Using Snapshot-based VMs. In *Proc. EuroSys*. 730–746.

[9] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, Mark L. Stillwell, David Goltzsche, Dave Eyers, Rüdiger Kapitza, Peter Pietzuch, and Christof Fetzer. 2016. SCONE: Secure Linux Containers with Intel SGX. In *Proc. USENIX OSDI*. 689–703. TODO verify.

[10] Algirdas Avižienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. 2004. Basic Concepts and Taxonomy of Dependable and Secure Computing. *IEEE Trans. Dependable and Secure Computing* 1, 1 (2004), 11–33. TODO verify.

[11] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1 (2004), 35–53.

[12] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. 2023. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334. IETF. TODO verify.

[13] Francesco Buccafurri, Vincenzo De Angelis, and Sara Lazzaro. 2024. MQTT-I: Achieving End-to-End Data Flow Integrity in MQTT. *IEEE Transactions on Dependable and Secure Computing* 21, 5 (2024), 4717–4734. https://doi.org/10.1109/TDSC.2024.3358630 TODO verify.

[14] Mike Burrows. 2006. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In *Proc. USENIX OSDI*. 335–350. TODO verify.

[15] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: Skip Redundant Paths to Make Serverless Fast. In *Proc. EuroSys*. 32:1–32:15.

[16] Cloudflare. 2022. Introducing Cloudflare Queues. Cloudflare blog, https://blog.cloudflare.com/introducing-cloudflare-queues/. TODO verify.

[17] Marcin Copik, Marcin Chrapek, Larissa Schmid, Alexandru Calotoiu, and Torsten Hoefler. 2024. SeBS-Flow: Benchmarking Serverless Cloud Function Workflows. arXiv preprint. TODO verify authors, venue and year.

[18] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefler. 2021. SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing. In *Proc. ACM/IFIP Middleware*. 64–78.

[19] Victor Costan and Srinivas Devadas. 2016. Intel SGX Explained. IACR Cryptology ePrint Archive, 2016/086.

[20] Flaviu Cristian. 1991. Understanding Fault-Tolerant Distributed Systems. *Commun. ACM* 34, 2 (1991), 56–78. TODO verify.

[21] Nilanjan Daw, Umesh Bellur, and Purushottam Kulkarni. 2020. Xanadu: Mitigating Cascading Cold Starts in Serverless Function Chain Deployments. In *Proc. ACM/IFIP Middleware*. 356–370. TODO verify.

[22] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80.

[23] John R. Douceur. 2002. The Sybil Attack. In *Proc. Int. Workshop on Peer-to-Peer Systems (IPTPS)*. 251–260.

[24] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall.

[25] Vajiheh Farhadi, Fidan Mehmeti, Ting He, Thomas F. La Porta, Hana Khamfroush, Shiqiang Wang, Kevin S. Chan, and Konstantinos Poularakis. 2021. Service Placement and Request Scheduling for Data-Intensive Applications in Edge Clouds. *IEEE/ACM Transactions on Networking* 29, 2 (2021). TODO verify.

[26] Yuqi Fu, Ruizhe Shi, Haoliang Wang, Songqing Chen, and Yue Cheng. 2024. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In *Proceedings of the USENIX Annual Technical Conference (ATC)*. 19–36. TODO verify.

[27] Alexander Fuerst and Prateek Sharma. 2021. FaaSCache: Keeping Serverless Computing Alive with Greedy-Dual Caching. In *Proc. ACM ASPLOS*. 386–400.

[28] Google Cloud. 2026. Configure event-driven function retries. Cloud Run functions documentation, https://docs.cloud.google.com/functions/docs/writing/retries. TODO verify; accessed October 2026.

[29] Naohiro Hayashibara, Xavier Défago, Rémy Yared, and Takuya Katayama. 2004. The φ Accrual Failure Detector. In *Proc. IEEE Symp. on Reliable Distributed Systems (SRDS)*. 66–78.

[30] Benjamin Holmes, Jason Waterman, and Dan Williams. 2024. SEVeriFast: Minimizing the Root of Trust for Fast Startup of SEV Microvms. In *Proc. ACM ASPLOS*. TODO verify title and pages.

[31] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. 2017. Gray Failure: The Achilles' Heel of Cloud-Scale Systems. In *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*. 150–155. TODO verify.

[32] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.

[33] Hans Niklas Jacob, Christian Werling, Robert Buhren, and Jean-Pierre Seifert. 2023. faulTPM: Exposing AMD fTPMs' Deepest Secrets. In *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*. TODO verify.

[34] Bahman Javadi, Derrick Kondo, Alexandru Iosup, and Dick Epema. 2013. The Failure Trace Archive: Enabling the Comparison of Failure Measurements and Models of Distributed Systems. *J. Parallel and Distributed Computing* 73, 8 (2013), 1208–1223.

[35] Ibrahim Kettaneh, Sreeharsha Udayashankar, Ashraf Abdel-hadi, Robin Grosman, and Samer Al-Kiswany. 2020. Falcon: Low Latency, Network-Accelerated Scheduling. In *Proceedings of the 3rd P4 Workshop in Europe (EuroP4)*. https://doi.org/10.1145/3426744.3431322 TODO verify author list.
[36] Dave Levin, John R. Douceur, Jacob R. Lorch, and Thomas Moscibroda. 2009. TrInc: Small Trusted Hardware for Large Distributed Systems. In *Proc. USENIX NSDI*. 1–14.
[37] Dingji Li, Zeyu Mi, Chenhui Ji, Yifan Tan, Binyu Zang, Haibing Guan, and Haibo Chen. 2023. Bifrost: Analysis and Optimization of Network I/O Tax in Confidential Virtual Machines. In *Proc. USENIX ATC*. TODO verify.
[38] Microsoft. 2026. Resilient Event Hubs and Functions design. Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/serverless/event-hubs-functions/resilient-design. TODO verify; accessed October 2026.
[39] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. 2025. Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX. In *Proc. ACM SIGMETRICS*. TODO verify.
[40] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *Proc. USENIX ATC*. 57–70.
[41] OASIS. 2019. *MQTT Version 5.0, OASIS Standard*. Technical Report. OASIS. TODO verify.
[42] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. 2013. Sparrow: Distributed, Low Latency Scheduling. In *Proc. ACM SOSP*. 69–84.
[43] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115. TODO verify.
[44] Thomas Rausch, Alexander Rashed, and Schahram Dustdar. 2021. Optimized Container Scheduling for Data-Intensive Serverless Edge Computing. *Future Generation Computer Systems* 114 (2021), 259–271. TODO verify.
[45] Nabil Schear, Patrick T. Cable, Thomas M. Moyer, Bryan Richard, and Robert Rudd. 2016. Bootstrapping and Maintaining Trust in the Cloud. In *Proc. Annual Computer Security Applications Conf. (ACSAC)*. 65–77.
[46] Carlos Segarra, Tobias Pfandzelter, et al. 2024. Serverless Confidential Containers: Challenges and Opportunities. In *Proc. Workshop on Serverless Systems, Applications and Methodology (SESAME)*. TODO verify authors.
[47] Glenn Shafer and Vladimir Vovk. 2008. A Tutorial on Conformal Prediction. *J. Machine Learning Research* 9 (2008), 371–421.
[48] Mohammad Shahrad, Rodrigo Fonseca, Iñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *Proc. USENIX ATC*. 205–218.
[49] Lujie Tang, Minxian Xu, Chengzhong Xu, and Kejiang Ye. 2024. Collaborative Resource Management and Workloads Scheduling in Cloud-Assisted Mobile Edge Computing across Timescales. arXiv:2405.20560. TODO verify; check for a journal version.
[50] Trusted Computing Group. 2019. Trusted Platform Module Library Specification, Family 2.0. TCG specification. TODO verify revision.
[51] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale Cluster Management at Google with Borg. In *Proc. EuroSys*. 18:1–18:17.
[52] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung, and Paulo Verissimo. 2013. Efficient Byzantine Fault-Tolerance. *IEEE Trans. Computers* 62, 1 (2013), 16–30.
[53] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proc. ACM SOSP*. 724–739.

A REPRODUCTION

Every result in this paper can be regenerated from raw logs. Table 7 lists what bit-for-bit reproduction needs. We checked it three ways. Before the NumPy pin, runners with and without AVX-512 disagreed on every run of one experiment (0 of 50 matched). After the pin, a probe experiment gave identical hashes on an Intel Xeon Platinum 8573C and on AMD EPYC 9V74, 9V45 and 7763 (the last without AVX-512), under Ubuntu 22.04 and 24.04. And the full sweep, re-executed on independently provisioned runners, matched 2,366 of 2,366 runs, with identical tables.
With the lockfile installed, the following regenerates the runs, compares them with the recorded hashes and rebuilds this document.

```
uv sync --frozen
make paper          # 21 specs x 10 seeds
make paper-verify   # compare hashes
voss repro runs/E09/<run>/manifest.json
```

Table 7. Requirements to reproduce the results bit for bit.

CPU	x86-64 with fused multiply-add (every Intel and AMD CPU since 2013). AVX-512 is neither required nor used: the pin selects the SSE4 baseline, so AVX2 and AVX-512 machines agree.
Operating system	Linux with the GNU C library (tested: Ubuntu 22.04 and 24.04, glibc 2.35 and 2.39).
Software	Python 3.11 and the dependency versions in the committed lockfile (NumPy 2.x, SciPy with HiGHS, pandas, PyArrow); `NPY_DISABLE_CPU_FEATURES="X86_V4 X86_V3 AVX512_ICL AVX512_SPR"`, set by the package, the Makefile and the CI workflows.
Special hardware	None: no TPM, reboot or BIOS access. The calibration bundle is committed.
Compute	21 experiments, 2,366 runs. About four hours as 21 parallel jobs on hosted runners; compressed raw logs are 0.1–0.7 GB per experiment.
Not covered	ARM, other operating systems and other C libraries are not bit-comparable; for them the portable claim is statistical (ten paired seeds). Wall-clock columns vary by machine and are excluded from the hashes.

```
make manuscript
```

B HYPOTHESIS SUMMARY

Table 8 lists every hypothesis, stated before the runs, with its verdict by the rules of Section 8.3. A dash marks cost-side information without a verdict.

Table 8. Hypotheses, paired relative change with 95% bootstrap interval, and verdict.

H	Claim	Exp.	Cell	Δ rel. [95% CI]	Verdict
H1	MILP planner cuts sync P99 vs no planning	E04	churn_med	-54.5% [-105.8, -12.6]	**supported**
H1	MILP planner cuts sync P99 vs no planning	E04	churn_high	-62.5% [-100.3, -27.0]	**supported**
H1	... and vs keep-alive TTL	E04	churn_med	-28.7% [-57.6, -7.7]	**supported**
H1	... and vs keep-alive TTL	E04	churn_high	-60.2% [-124.6, -12.2]	**supported**
H1	donor memory held warm: MILP vs keep-alive TTL (cost side)	E04	churn_med	+109.8% [+90.2, +125.8]	–
H2	f-robust plans keep success where non-robust collapse (α=0.2 mixed)	E08	mixed α=0.2	-0.1% [-0.3, +0.0]	inconclusive
H3	Pareto front has a knee; moderate exposure loses little latency	E23		+1.6%	**supported**
H4	progress detection beats heartbeat-only on success under α=0.2	E08	mixed α=0.2	-3.2% [-4.3, -2.1]	**supported**
H5	appliance warm start ≈ Firecracker without attestation (L3)	E-CS1 (sim)		-2.0%	**supported**
H5	appliance snapshot start ≈ Firecracker without attestation (L2)	E-CS1 (sim)		+2.5%	**supported**
H5	CVM classes are 10-100x slower than the appliance to establish trust	E-CS1 (sim)		-92.7%	**refuted**
H6	GDSF + P2P + HDD tier beat LRU without P2P/HDD (sync P99)	E09		+2.8% [-2.9, +11.9]	inconclusive
H6	... GDSF vs LRU alone	E09		+1.7% [-0.2, +4.1]	inconclusive
H7	geo clusters reduce sync P99 vs one global scheduler (EU)	E05	regions=[eu]	+17.3% [+12.5, +22.0]	**refuted**
H7	... EU+NA	E05	regions=[eu, na]	+16.6% [+5.2, +33.1]	**refuted**
H7	... world	E05	regions=[]	+4.5% [+0.9, +8.7]	**refuted**
H8	planner MILP beats greedy by ≥5% sync P99 (high churn)	E04	churn_high	-8.1% [-20.8, +1.1]	inconclusive
H9	distribution-aware (quantile) beats expectation-only on P99 at high churn	E03	churn_high, name=blr	-2.8% [-5.3, -0.8]	weak
H10	accounting is off the critical path: settlement period 10 s vs 300 s	E13	faults=[]	+1.5% [-0.2, +4.2]	**supported**
H10	... receipt batch 0.2 s vs 5 s	E13	faults=[]	-1.0% [-3.3, +1.0]	**supported**
H11	simple baseline matches VOSS when homogeneous, stable and benign	E10	hw=C1	+63.7% [+25.0, +112.8]	**refuted**
H-FT1	progress detection vs heartbeat-only: success with a black-holed scheduler	E15	faults=sched_blackhole	+3.2% [+2.7, +3.7]	**supported**
H-FT3	risk-proportional hedging (θ=.3, 5%) cuts sync P99 under α=0.1	E16	mixed α=0.1	-13.3% [-32.5, +1.4]	inconclusive
H-FT3	... at what extra-attempt cost	E16	mixed α=0.1	+253.0% [+182.6, +332.0]	–
RQ15	VOSS vs Lambda-like reference, sync P99 at 20 inv/s	E17	rate=20	+14.1% [+10.4, +18.3]	–
E12	reputation tie-break (beta-decay) improves sync P99 vs no reputation	E12		-2.4% [-4.4, -0.8]	weak
E12	... and success rate	E12		-0.0% [-0.0, +0.0]	inconclusive